BASIC GUIDE

The Kalman Filter

Estimating what you cannot see, one measurement at a time

Intuition · one model · one algorithm · many applications

Pictures first, few formulas, one notation throughout: state x, measurement z, model A and H, noise Q and R, uncertainty P, gain K.

Three ideas carry the whole lecture

Everything later — tracking, hedge ratios, nowcasting — reuses these three ideas

1

Two noisy guesses beat one

Blend a model prediction with a measurement, weighted by how much you trust each.

2

One model with two equations

A state equation says how the hidden state moves; a measurement equation says what the sensor sees.

3

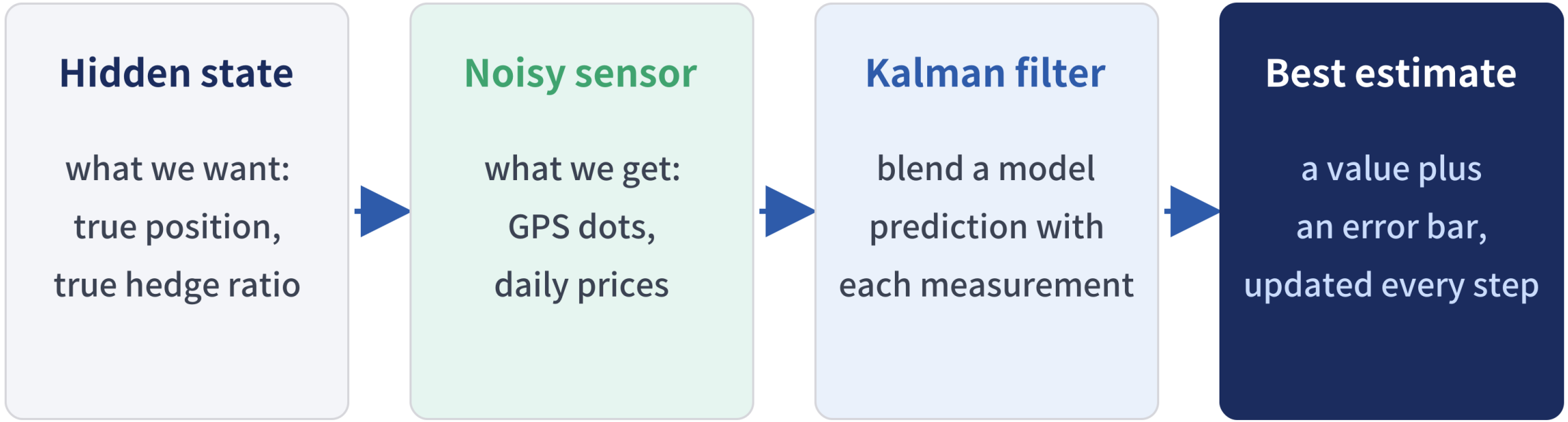
One loop with two steps

Predict, then update — repeat at every new data point. The gain K sets the blend.

Learn the template once; each application only fills in x, A, z, H, Q and R differently.

The filter infers a hidden state from noisy measurements

Basic knowledge — what problem the Kalman filter solves



The filter never sees the hidden state directly. It infers it.

01

ORIGINS

Where the Kalman Filter Came From

From planet orbits to the Apollo guidance computer

Kalman turned least squares into a step-by-step update

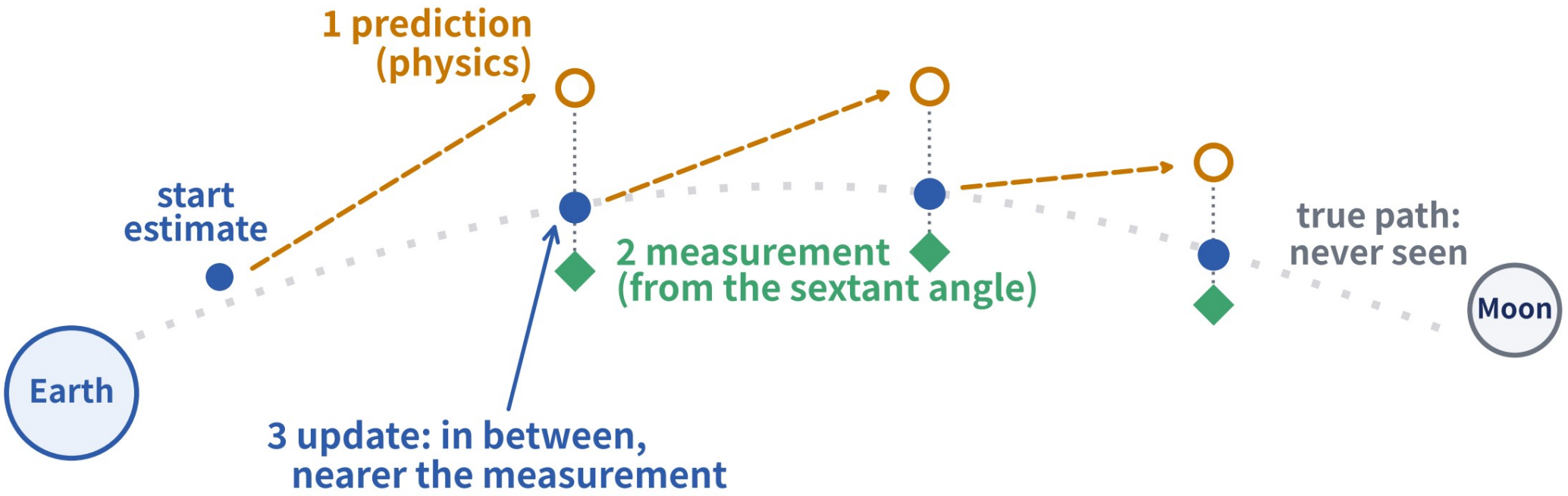
Rudolf Kálmán published the recursive filter in 1960; Stanley Schmidt at NASA Ames put it on Apollo



Kalman's step: do not refit everything — correct the last estimate

Apollo corrected its predicted position with each sextant sighting

Ring = physics prediction · diamond = what the sighting says · blue = updated position in between



Each new start = the last update. Blue points follow the hidden path closely.

02

INTUITION

Blending Two Noisy Guesses

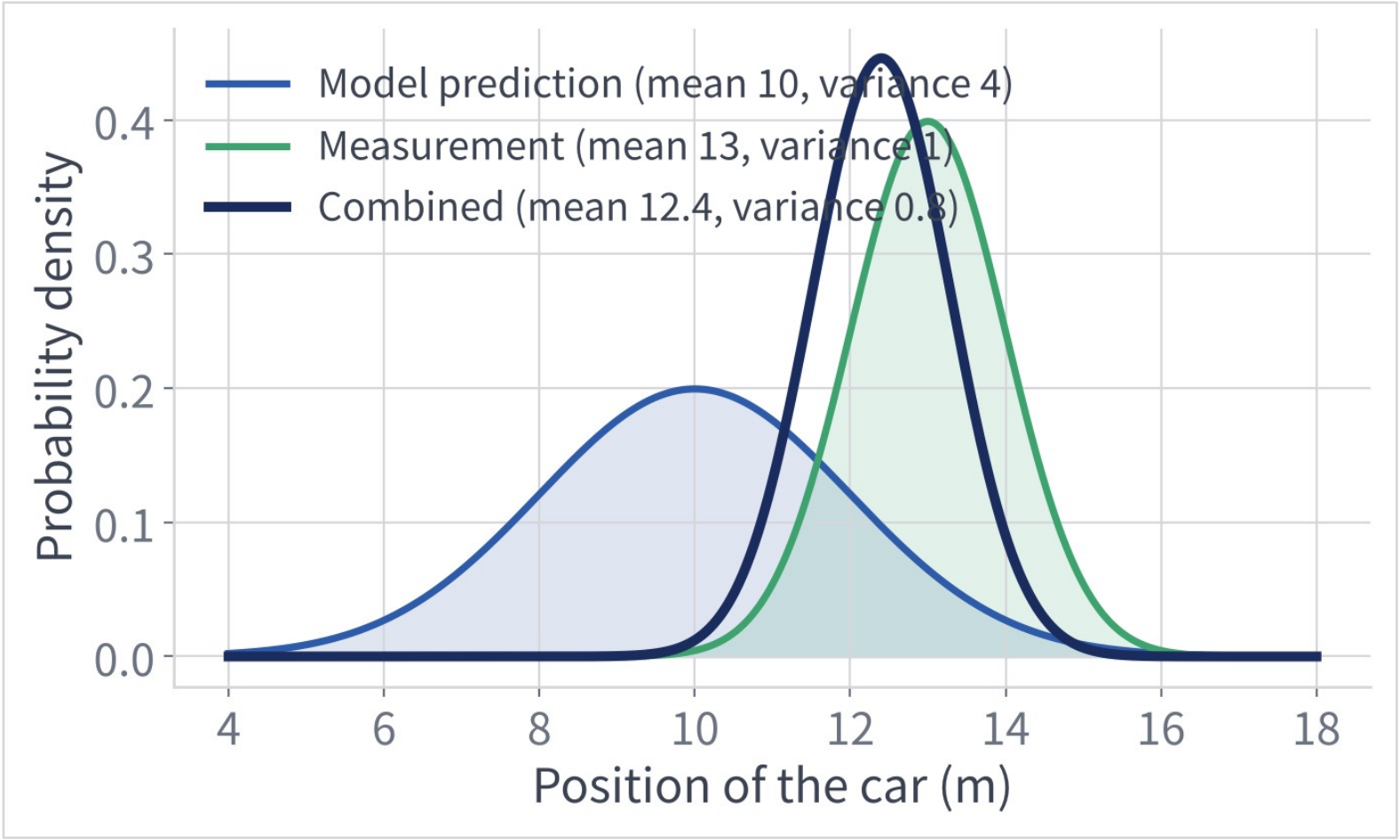
Why the combination is more precise than either piece

Combining two noisy guesses gives a sharper answer

The model says 10 m (variance 4); GPS says 13 m (variance 1)

Read it like this

- Each bell curve is one guess
- The narrower curve is trusted more
- The result sits closer to GPS
- The result is narrower than both

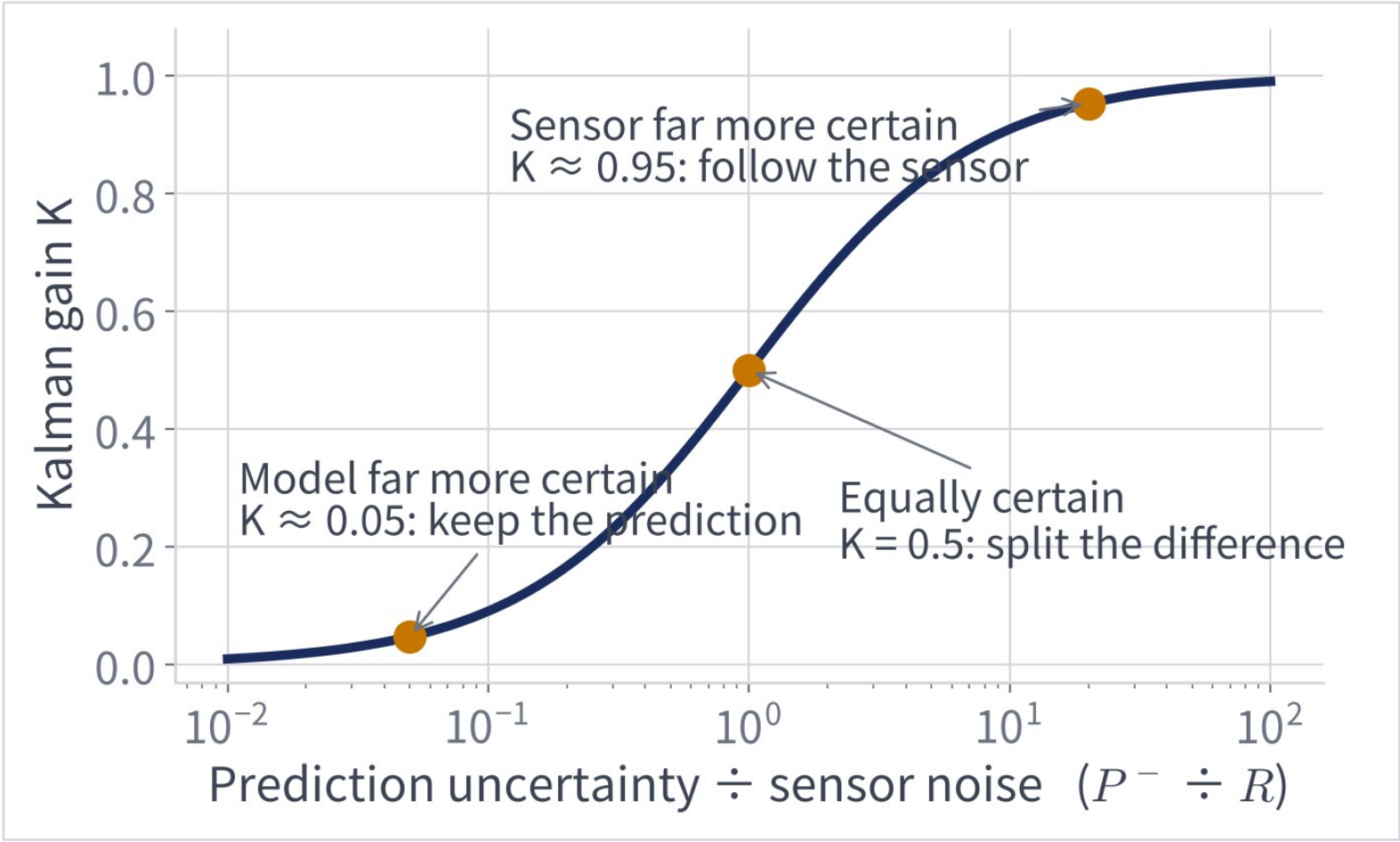


The Kalman gain is a trust dial between model and sensor

K runs from 0 (ignore the sensor) to 1 (copy the sensor)

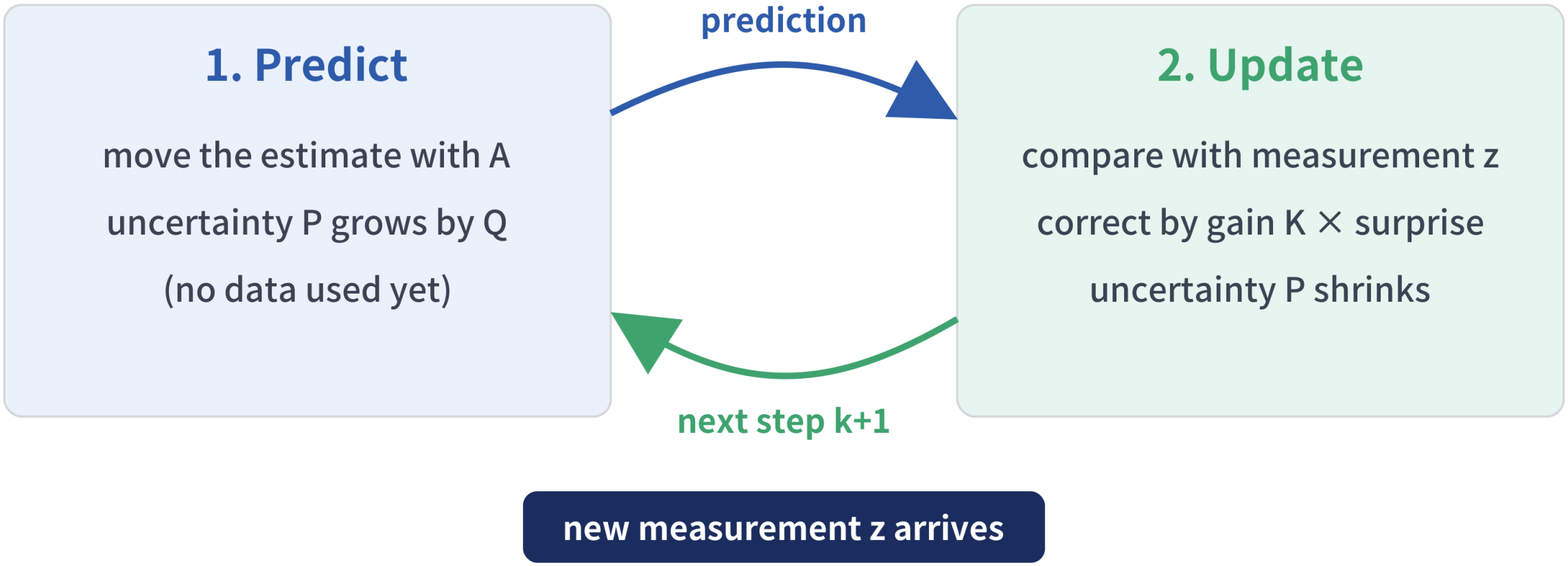
What moves the dial

- Model unsure (large P^-): K up
- Sensor noisy (large R): K down
- Only the ratio matters



The filter alternates between predicting and updating

Predict with the model, then correct with the measurement — at every time step k

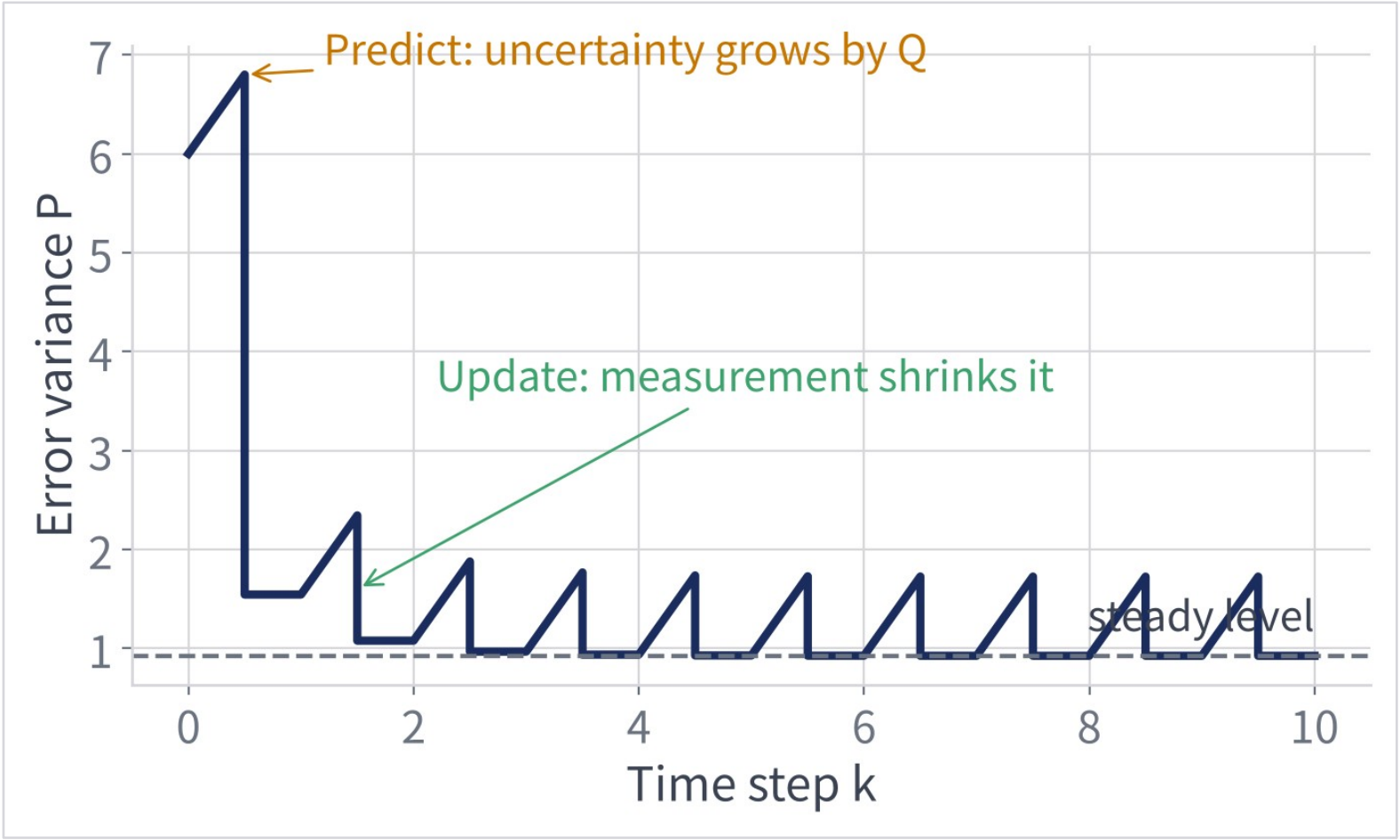


Uncertainty rises at each prediction and falls at each update

Error variance P traced over ten steps with fixed Q and R

Why it matters

- Prediction adds Q
- Measurement removes part
- P settles at a steady level
- So K settles too



03

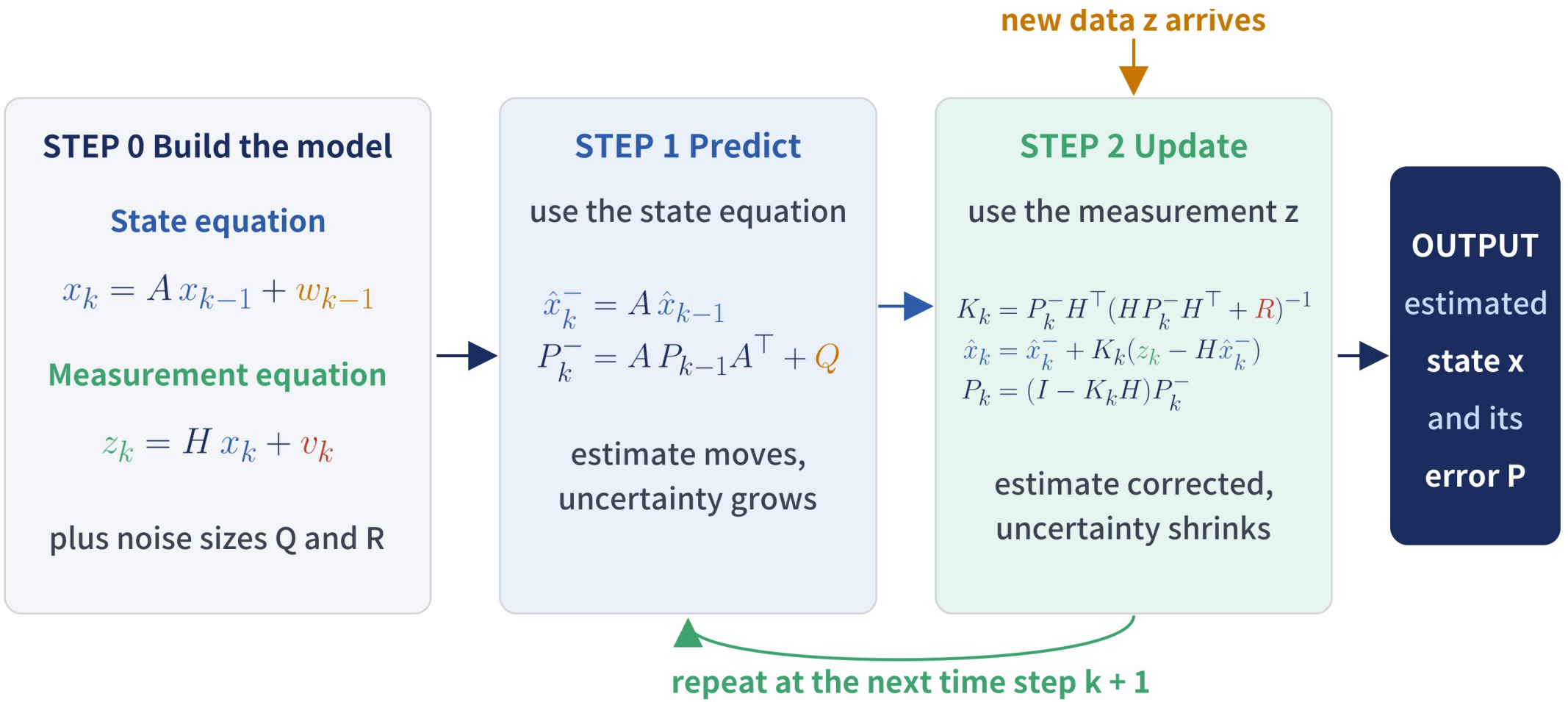
THE MODEL

The Basic State-Space Model

Two equations and two noise terms — the same template for every application

The Kalman filter is a model plus a two-step loop

Step 0 is done once; steps 1 and 2 repeat at every time step k



The state equation says how the hidden state moves

State transition equation — the model's own forecast of the next state

$$\underbrace{x_k}_{\text{state now}} = \underbrace{A\,x_{k-1}}_{\text{how it moves}} + \underbrace{w_{k-1}}_{\text{random shock}}$$

Read each piece

- A: how today follows yesterday
- w: what the model cannot foresee

Car example

- x = position and velocity
- A: new position = old + velocity times time
- w: bumps and small accelerations

The measurement equation says what the sensor sees

Measurement equation — links the hidden state to the observed data

$$\underbrace{z_k}_{\text{what we measure}} = \underbrace{H\,x_k}_{\text{what the sensor sees}} + \underbrace{v_k}_{\text{sensor noise}}$$

Read each piece

- H picks out the observed part
- v: sensor noise
- z is all we ever see

Car example

- GPS reports position only
- So H keeps position, drops velocity
- v: GPS error of a few metres

Two variances, Q and R, describe how wrong each side can be

Both noises are assumed normal with mean zero

$$w_k \sim \mathcal{N}(0,\, Q) \qquad v_k \sim \mathcal{N}(0,\, R)$$

Q: process noise

- How much the true state can wander
- Large Q: the model is loose

R: measurement noise

- How noisy the sensor is
- Large R: the data are poor

One notation is used from here to the last slide

Subscript k is the time step; a hat means an estimate; a minus sign means before the measurement

Symbol	Name	Car tracking	Pairs trading
x	state (hidden)	position, velocity	hedge ratio β
z	measurement	GPS position	EWC price
A	transition model	physics of motion	1 (β drifts slowly)
H	measurement model	keep position	EWA price
Q, R	process / sensor noise	bumps / GPS error	β drift / spread noise
P, K	uncertainty / gain	computed by the filter	computed by the filter

04

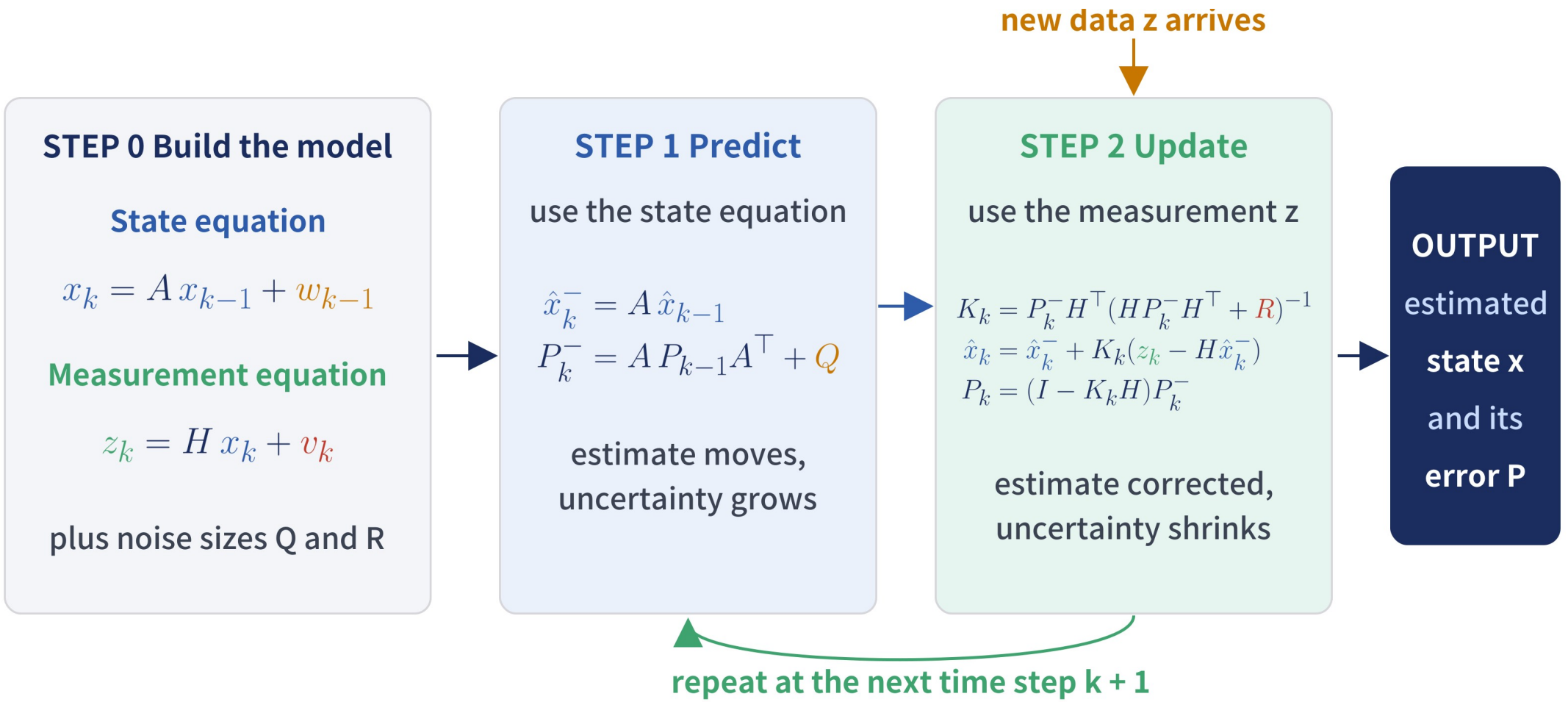
THE ALGORITHM

Predict and Update

Five lines that are the whole Kalman filter

The algorithm consists of the two middle boxes

With the model built, the filter only predicts and updates



The predict step pushes the estimate and its uncertainty forward

Uses the state equation only — no new data yet

$$\hat{x}_k^- = A\,\hat{x}_{k-1}$$

$$P_k^- = A\,P_{k-1}\,A^\top + Q$$

First line

- Move the last estimate with A
- No new data used yet

Second line

- Carry uncertainty forward with A
- Add Q: the future is less certain

The update step corrects the prediction with the measurement

Uses the measurement equation — compute K, correct, shrink P

$$K_k = P_k^- H^\top \left(H P_k^- H^\top + R \right)^{-1}$$
$$\hat{x}_k = \hat{x}_k^- + K_k \left(z_k - H \hat{x}_k^- \right)$$
$$P_k = (I - K_k H) P_k^-$$

Three lines

- Gain: how much to trust z
- Estimate: add K times the surprise
- Uncertainty: smaller after data

The surprise

- z minus the predicted measurement
- Called the innovation
- Zero surprise means no change

With a single number the gain is a simple fraction

Scalar case with H = 1 — the version used in most worked examples

$$K_k = \frac{P_k^-}{P_k^- + R} \qquad (\text{one number},\ H = 1)$$

Two limits

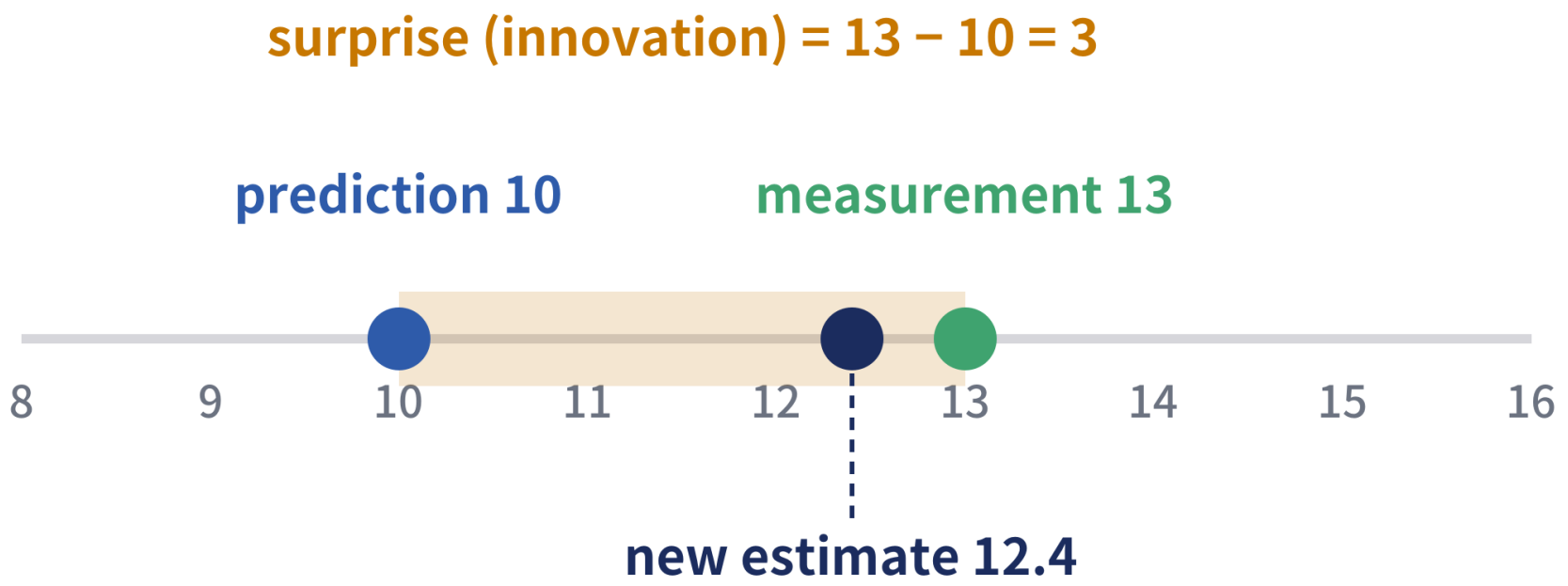
- R → 0: K → 1, copy the sensor
- P^- → 0: K → 0, keep the model

Same thing in words

- New estimate is a weighted average
- Weights are set by uncertainty

The estimate moves part of the way toward the measurement

Same numbers as the bell-curve slide: K = 4 ÷ (4 + 1) = 0.8



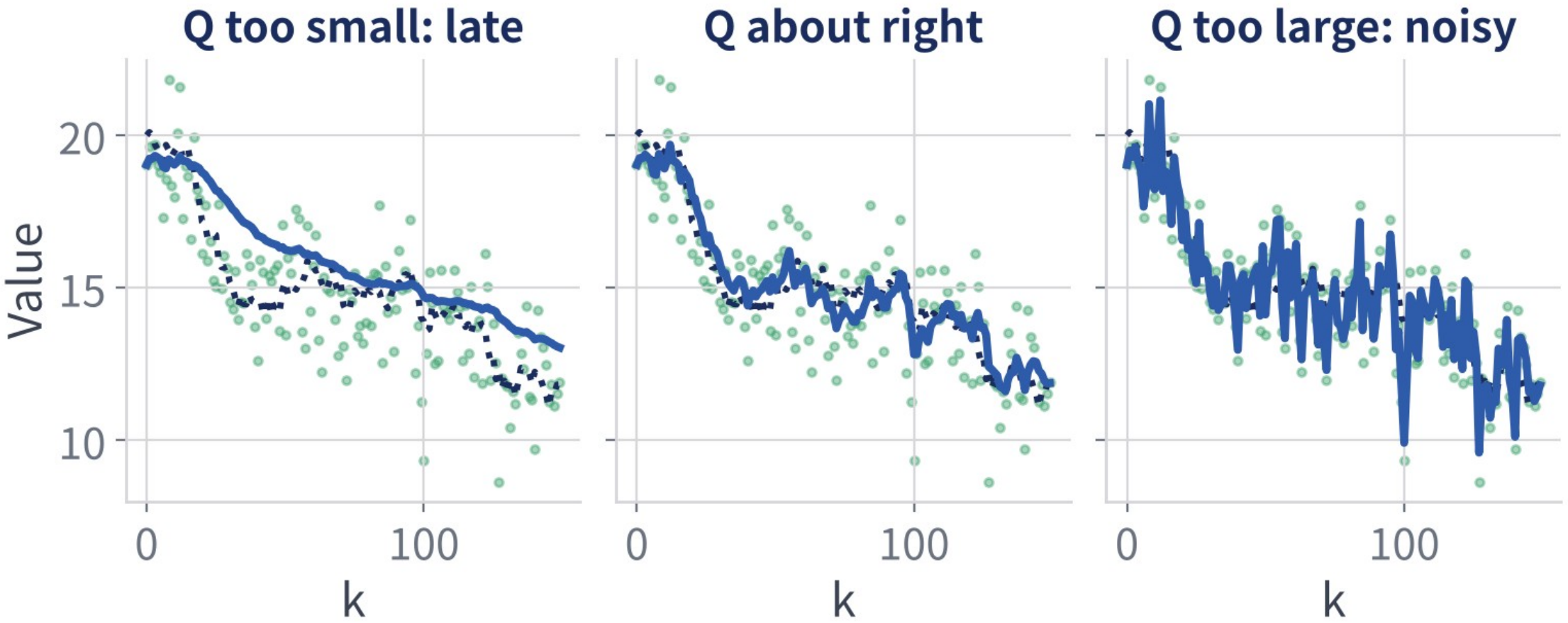
K = 0.8

move 80% of the surprise

10 + 0.8 × 3 = 12.4

The ratio of Q to R decides between smooth and responsive

Same data, three choices of Q with R fixed — dots are measurements, dotted line is the truth



Five lines of Python implement the whole filter

Scalar version of the predict and update equations

Python · the five equations, line by line

```
A, H, Q, R = 1.0, 1.0, 0.12, 1.96   # the model
x_hat, P = 0.0, 10.0                # first guess

for z in measurements:
    # 1. Predict
    x_pred = A * x_hat
    P_pred = A * P * A + Q

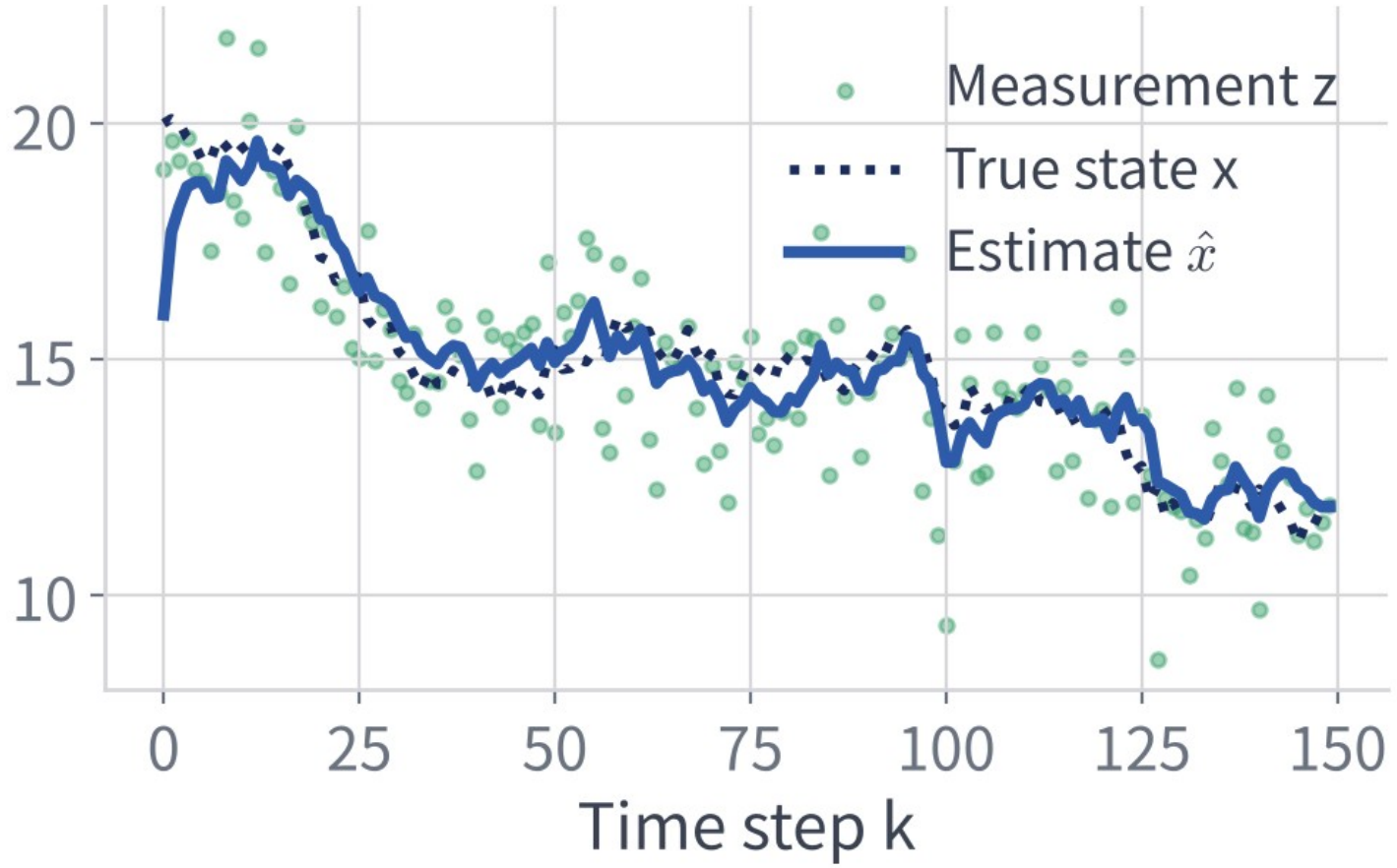
    # 2. Update
    K     = P_pred * H / (H * P_pred * H + R)
    x_hat = x_pred + K * (z - H * x_pred)
    P     = (1 - K * H) * P_pred
```

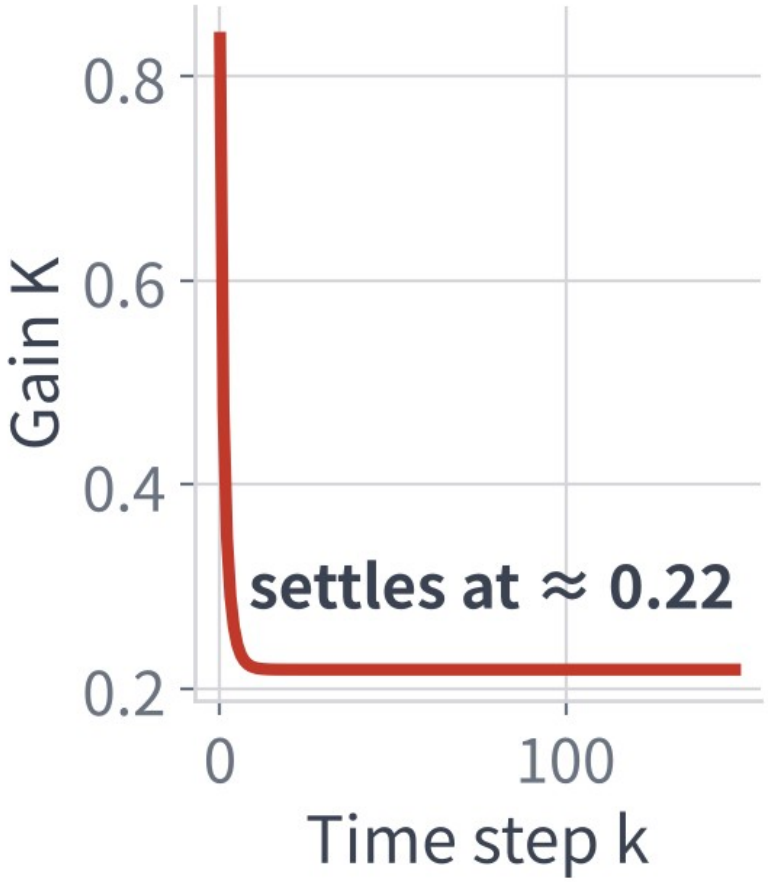
Match to the slides

- Lines 6–7 = predict step
- Lines 10–12 = update step
- Matrices: use @ and .T
- Libraries: filterpy, pykalman

The code tracks the truth and its gain settles quickly

Output of the code on simulated data: error falls from 1.4 for raw data to about 0.6





Every application is built by answering five questions

Use this checklist for each case in the next section

1. State x

What is hidden?

The quantity you want, which you never see directly.

β, trend, position

2. Model A

How does it move?

Constant, random walk, or physics.

A = 1 is common

3. Sensor H

What is seen?

How the state maps to the measurement z.

H can change daily

4. Noise Q, R

How wrong?

Set by judgement or estimated from data.

only Q/R matters

5. Start

First guess

A rough value with a large P.

data take over fast

05

APPLICATIONS

Applications

Same five equations; only the ingredients change

Seven applications differ only in what fills each slot

Read across a row: that row is a complete Kalman filter

Application	State x	Model A	Measurement z	Sensor H
Car tracking	position, velocity	physics	GPS position	[1 0]
Sensor fusion	position	random walk	GPS and wheel	both sensors
Price trend	level, slope	trend	daily price	[1 0]
Pairs hedge ratio	β	1	EWC price	EWA price
Stock beta	α, β	1	stock return	[1, market]
Nowcasting	growth gap G	φ	released data	1 (or none)
Yield curve	factor gaps F	0.97	yields	factor loadings

Car tracking: state is position and velocity, GPS sees position

Model first — both equations of the car tracking filter

$$\text{State equation}: \quad \begin{bmatrix} \text{pos} \\ \text{vel} \end{bmatrix}_k = \begin{bmatrix} 1 & \Delta t \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \text{pos} \\ \text{vel} \end{bmatrix}_{k-1} + w_{k-1}$$

$$\text{Measurement equation}: \quad z_k^{\text{GPS}} = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \text{pos} \\ \text{vel} \end{bmatrix}_k + v_k$$

Slots

- x: position and velocity
- A: constant-speed physics
- H: GPS reads position only

Settings used

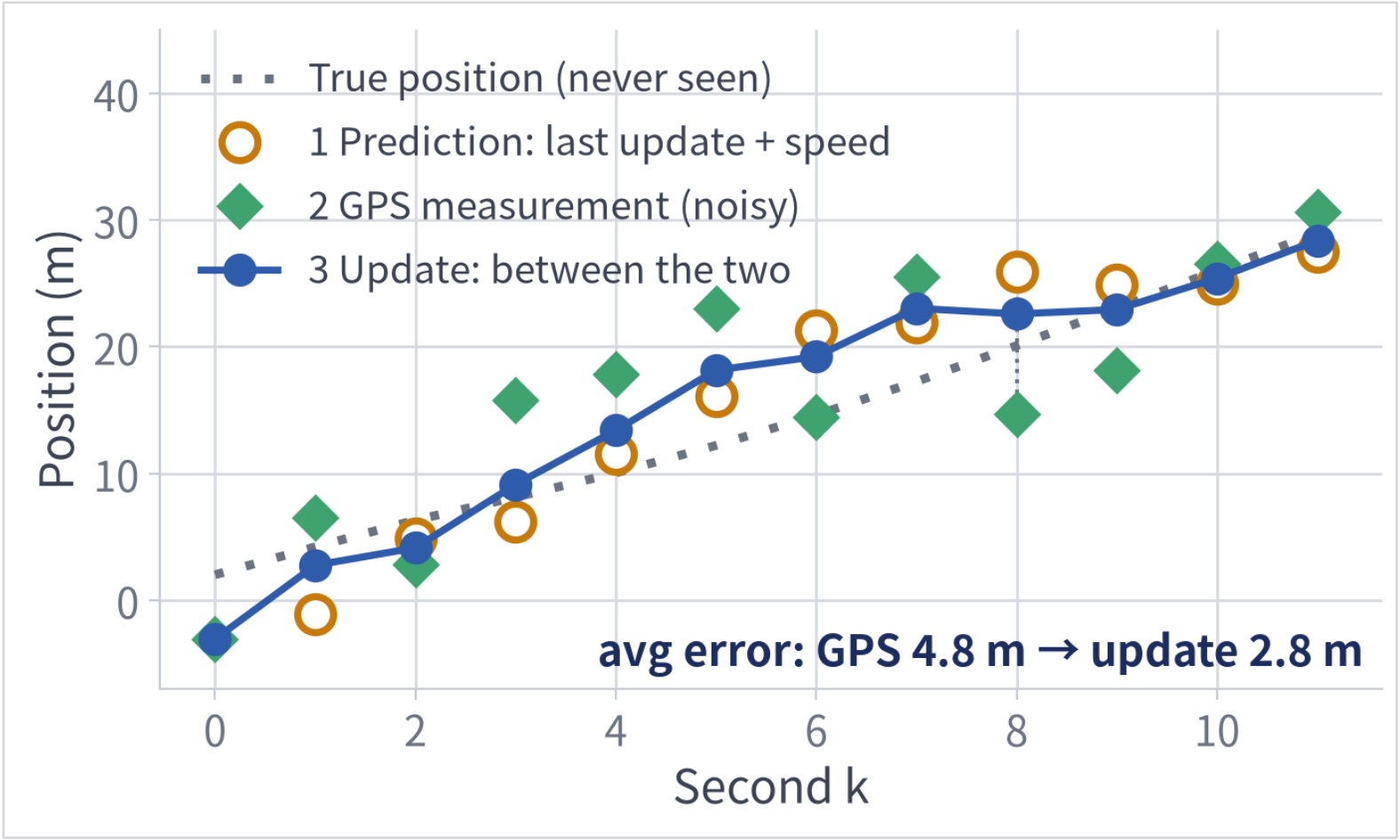
- Δt = 1 second
- GPS noise R = 36 (std 6 m)
- Q small: gentle accelerations

Each second the filter predicts, measures, then adjusts in between

The true position is never seen, yet the blue updates stay closer to it than the GPS readings

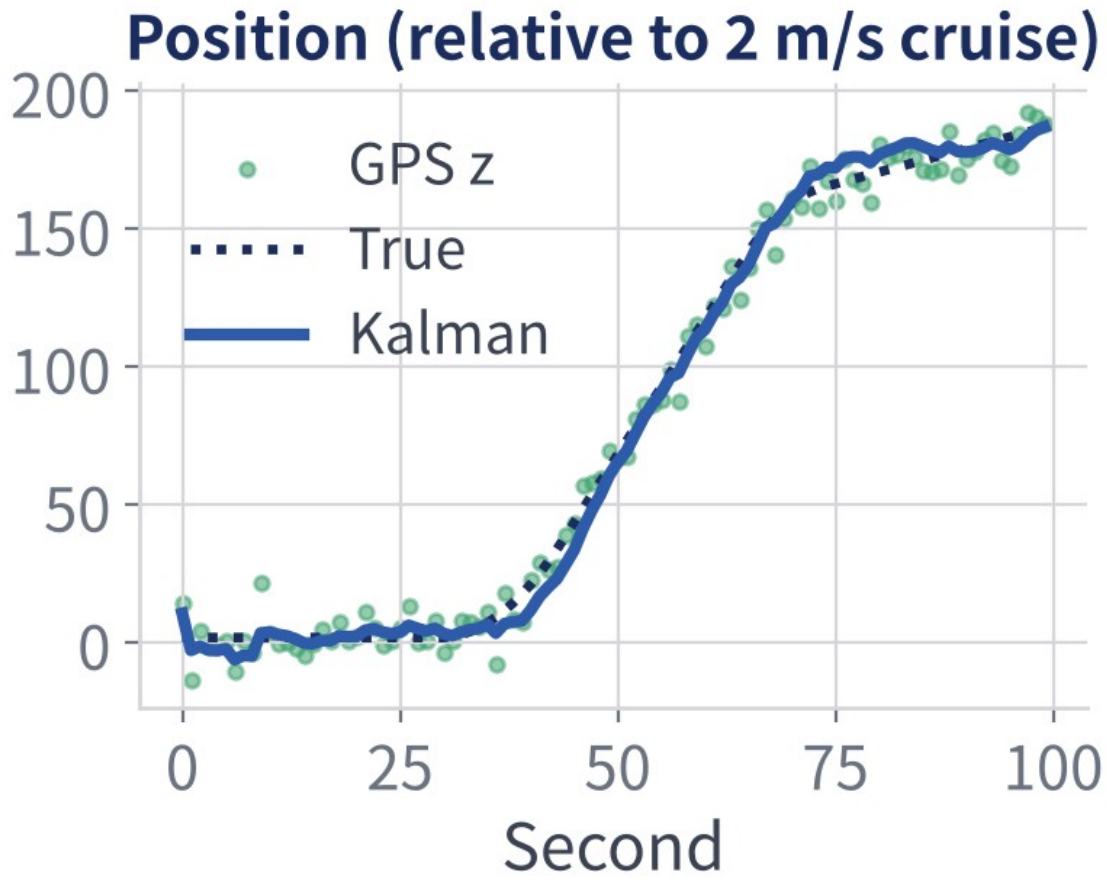
Each second

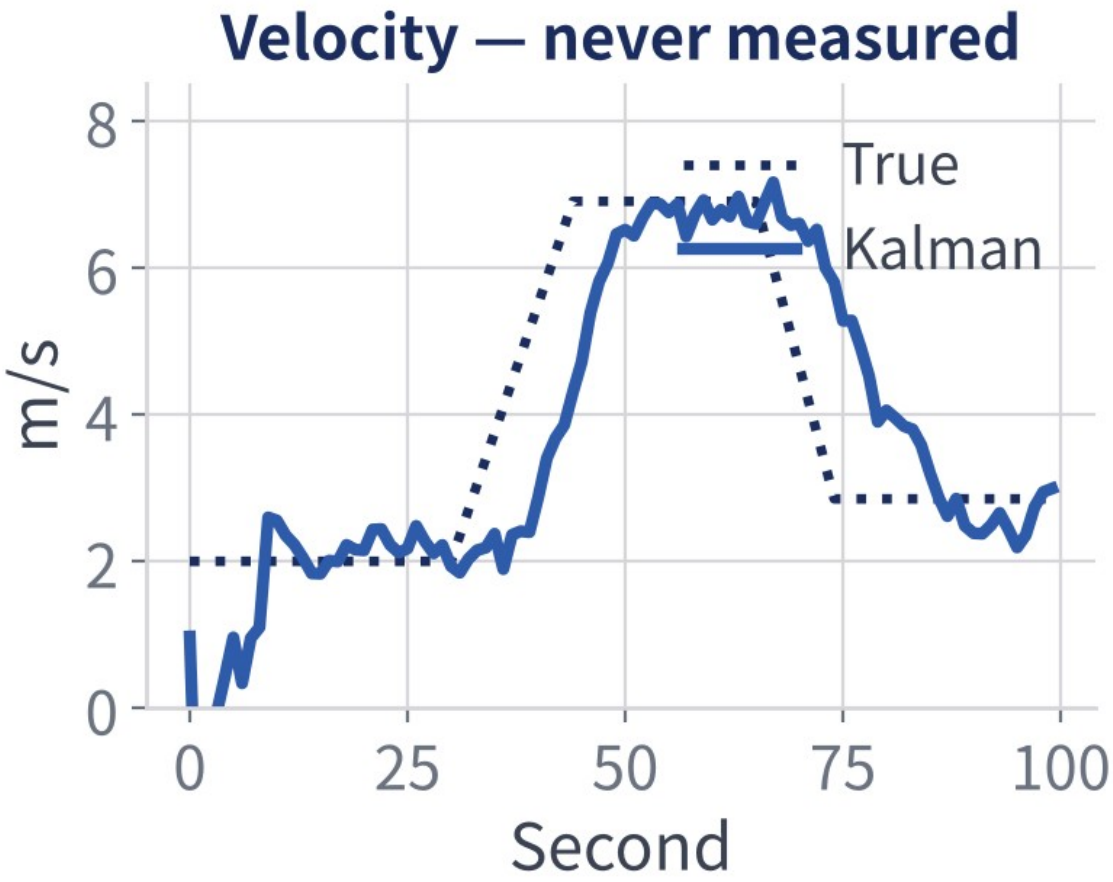
- 1 Ring: last update + speed
- 2 Diamond: GPS reading
- 3 Blue: move part way to GPS
- Blue error ≈ 0.6 × GPS



GPS alone is jumpy; the filter also recovers the speed

Car tracking with x = [position, velocity] — velocity is never measured



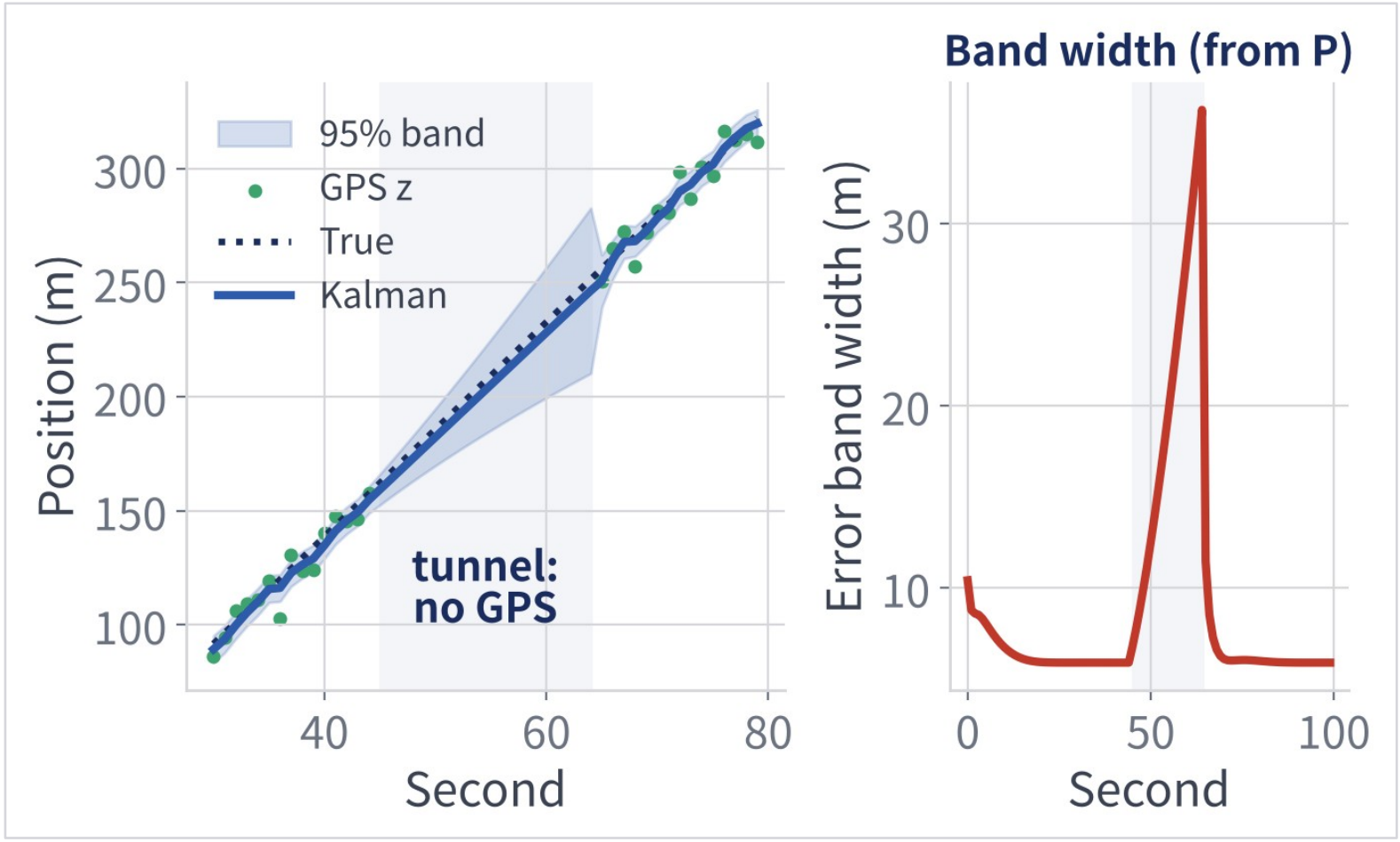


When GPS drops out, the filter predicts and widens its error band

Simulated drive: the GPS signal is lost between second 45 and 64 (grey area)

Second by second

- Grey: no GPS, so predict only
- Speed carries the estimate
- P grows: band widens
- GPS back: band shrinks



Inside the tunnel the filter keeps driving on its model

Same five equations; the update lines are simply not run while z is missing

No GPS means no update.
Position moves by velocity times Δt, and P grows by Q each second.

What the filter does

- Uses the last speed it estimated
- Moves the position forward
- Reports a wider error band

When GPS comes back

- P is large, so K is close to 1
- The first readings get big weight
- The band shrinks within seconds

Sensor fusion: one state, two measurement rows

Model first — GPS and wheel sensor both measure the same position

$$\text{State equation}: \quad \text{pos}_k = \text{pos}_{k-1} + w_{k-1}$$

$$\text{Measurement equation}: \quad \begin{bmatrix} z_k^{\text{GPS}} \\ z_k^{\text{wheel}} \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \text{pos}_k + \begin{bmatrix} v_k^{\text{GPS}} \\ v_k^{\text{wheel}} \end{bmatrix}$$

Slots

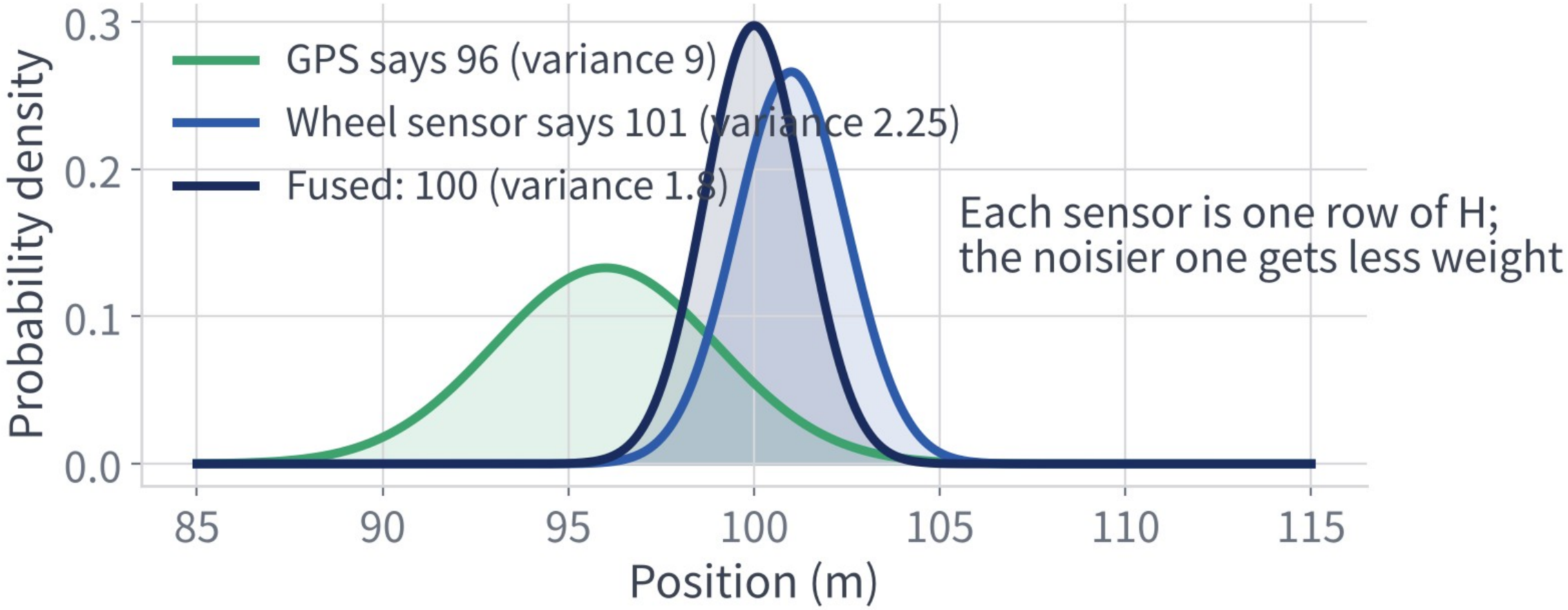
- x: position
- A = 1: random walk
- H: one row per sensor

Settings used

- Q = 0.09
- R: 9 for GPS, 2.25 for wheel
- each row updates in turn

Two sensors give two bells; the fused bell is narrower than both

Picture of the model: each sensor is one row of the measurement equation



With two measurements the Kalman gain becomes a row vector

Same gain formula as the update step; H now has two rows, R holds both sensor variances

$$H = \begin{bmatrix} 1 \\ 1 \end{bmatrix}, \quad R = \begin{bmatrix} R_1 & 0 \\ 0 & R_2 \end{bmatrix} \quad \Rightarrow \quad K_k = P_k^- H^\top \left(H P_k^- H^\top + R \right)^{-1} = \underbrace{\frac{P_k^-}{D} \begin{bmatrix} R_2 & R_1 \end{bmatrix}}_{1\times 2 \text{ row vector}}$$

$$D = P_k^- (R_1 + R_2) + R_1 R_2$$

$$\hat{x}_k = \hat{x}_k^- + K_1 \left(z_k^{\text{GPS}} - \hat{x}_k^- \right) + K_2 \left(z_k^{\text{wheel}} - \hat{x}_k^- \right)$$

$$P_k = (1 - K_1 - K_2)\, P_k^-$$

Read the gain

- K_1 is multiplied by R_2, K_2 by R_1
- Noisier sensor gets the smaller weight

Size of K

- Rows = states (1), columns = sensors (2)
- In general K is n states × m sensors

Updating with both sensors at once or in turn gives the same answer

True whenever the two sensor noises are independent, so that R is diagonal

Vector gain in one step = scalar gains in two steps.
The charts here use the two-step way: no matrix inverse needed.

All at once

- Stack both readings into z
- One gain K with two entries
- Needs the inverse of a 2 × 2 S

One after the other

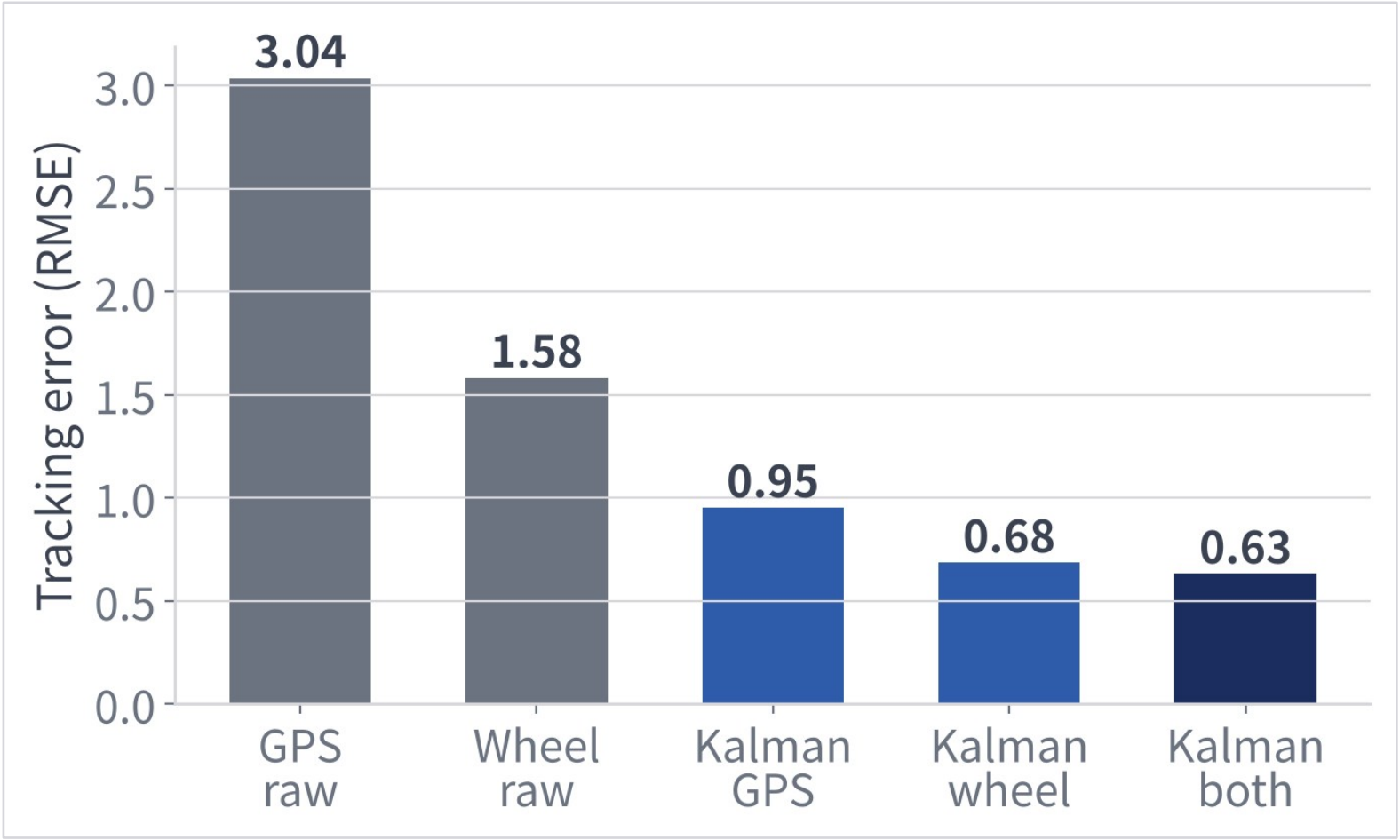
- Update with GPS, $R_1 = 9$
- Then with the wheel, $R_2 = 2.25$
- Each step is the scalar formula

Two sensors together beat the better sensor alone

Sensor fusion — each measurement updates the estimate with its own R

How fusion works

- Update with GPS (R = 9)
- Then wheel (R = 2.25)
- Noisier sensor gets less K
- Phones do this every second



Price trend: state is level and slope, price is level plus noise

Model first — the local linear trend model

$$\text{State equation}: \quad \begin{bmatrix} \text{level} \\ \text{slope} \end{bmatrix}_k = \begin{bmatrix} 1 & 1 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} \text{level} \\ \text{slope} \end{bmatrix}_{k-1} + w_{k-1}$$

$$\text{Measurement equation}: \quad \text{price}_k = \begin{bmatrix} 1 & 0 \end{bmatrix} \begin{bmatrix} \text{level} \\ \text{slope} \end{bmatrix}_k + v_k$$

Slots

- x: level and slope
- A: level grows by slope
- H: price shows the level

Settings used

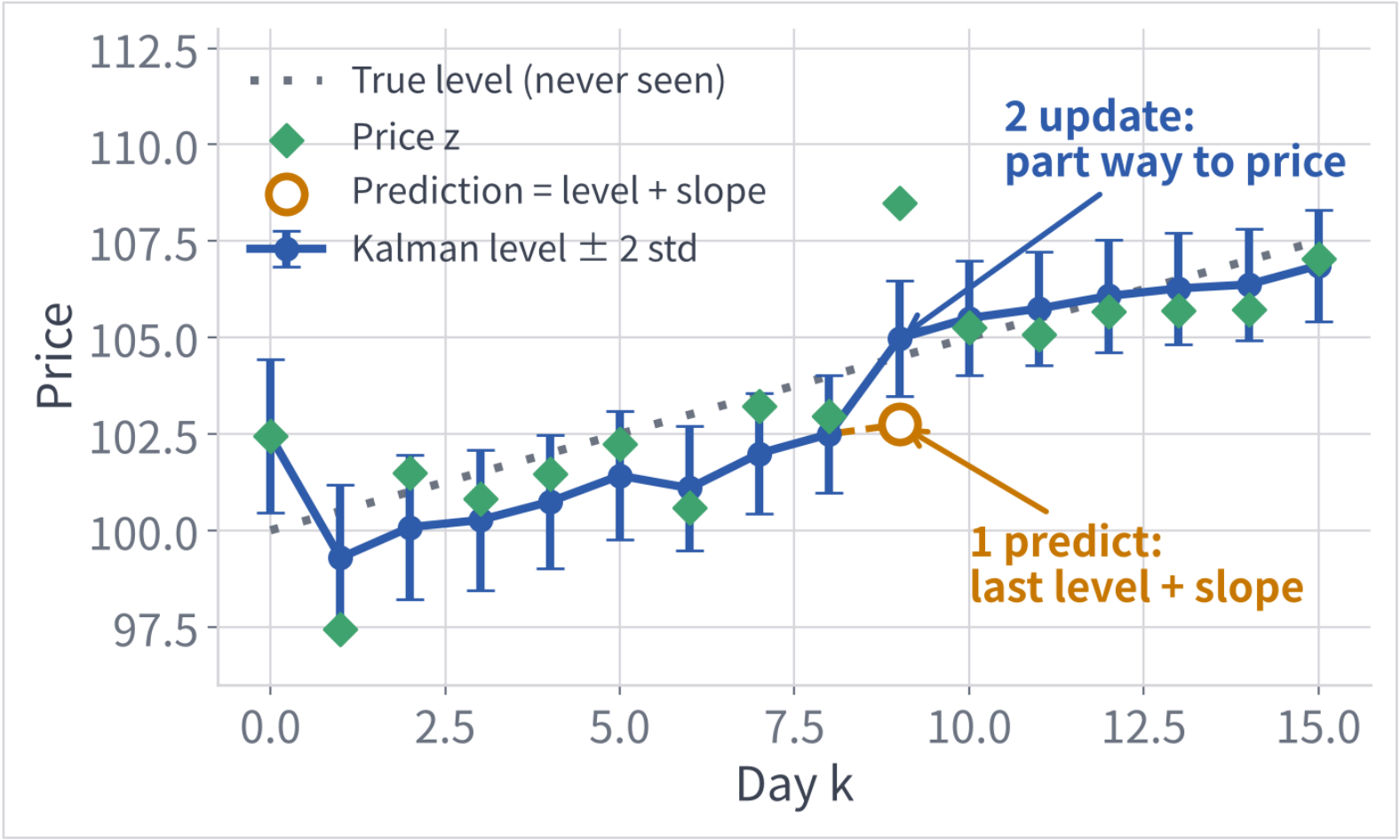
- Q: 0.05 level, 0.002 slope
- R = 2.56 (std 1.6)
- smoothness set by Q/R

Each day the filter predicts with the slope, then pulls toward the price

The true level is never seen; the circled day shows the two steps

How to read

- Dotted: true level
- Diamonds: price z
- Ring: level + slope
- Blue: after update
- Bars: ± 2 std

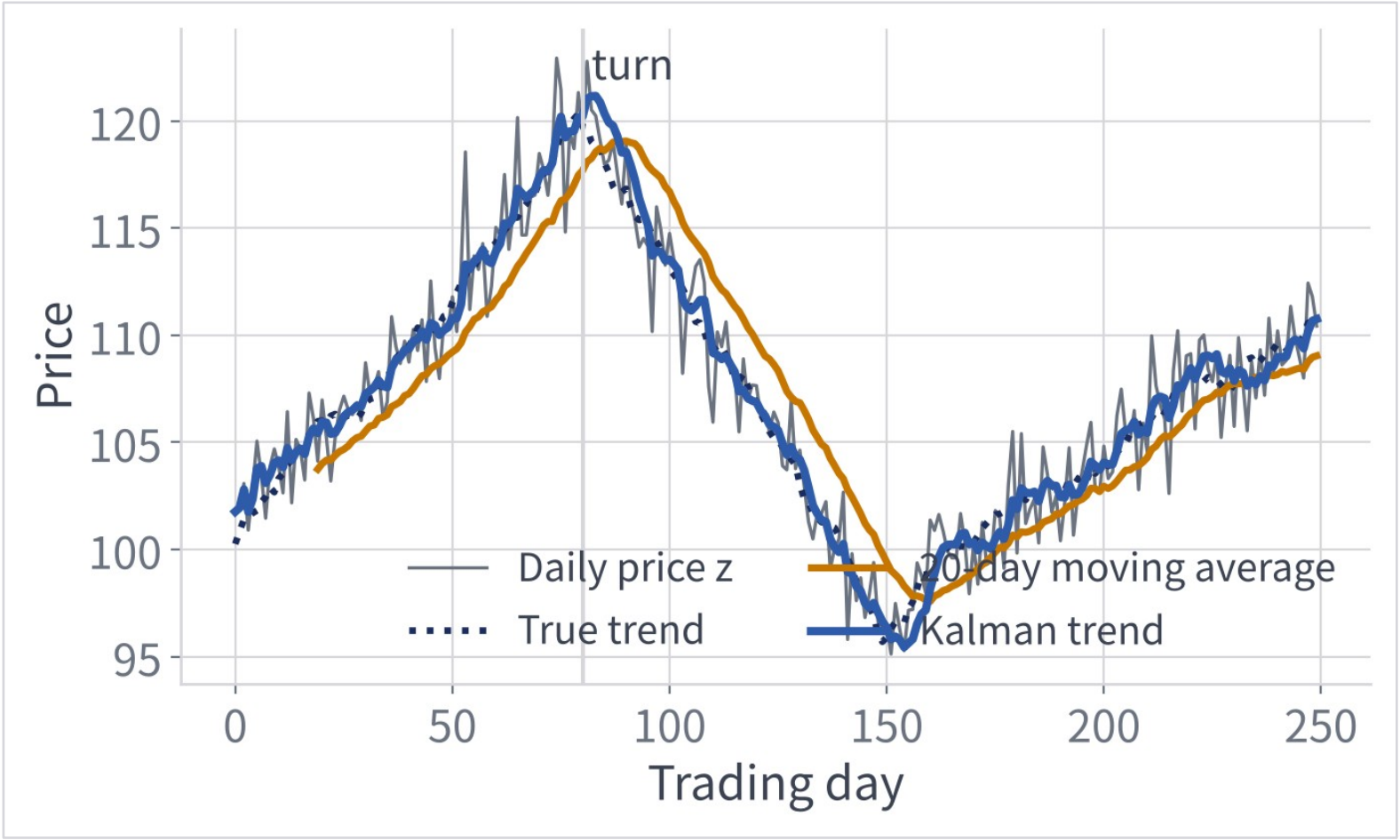


A Kalman trend turns faster than a moving average

Trend extraction with x = [level, slope] — the slope lets the model anticipate

Why less lag

- Average waits for old data
- Filter carries a slope
- Q/R sets smoothness
- No window to choose



Pairs trading: the hedge ratio is the state, EWA price is H

Model first — β drifts as a random walk; EWC price is the measurement

$$\text{State equation}: \quad \beta_k = \beta_{k-1} + w_{k-1}$$

$$\text{Measurement equation}: \quad \text{EWC}_k = \text{EWA}_k \cdot \beta_k + v_k$$

Slots

- x: hedge ratio β
- A = 1: β drifts slowly
- H: today's EWA price

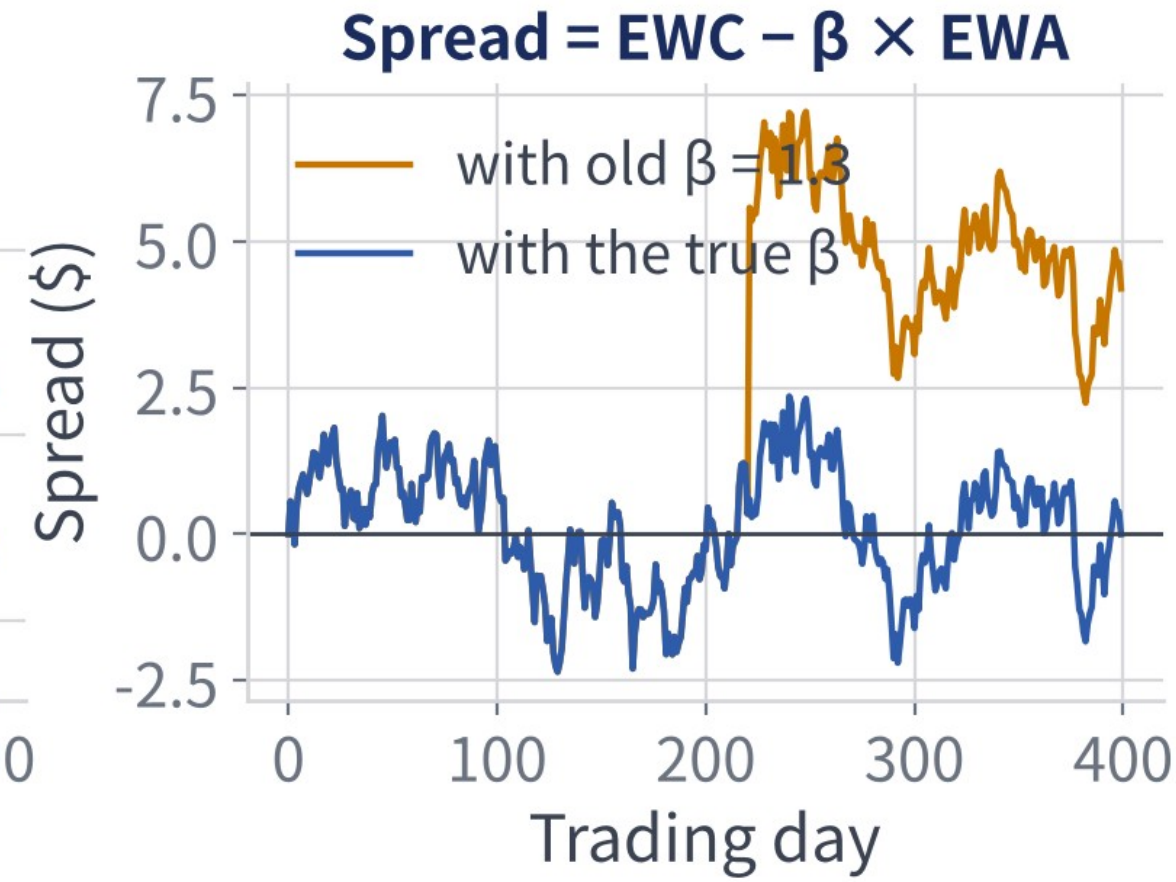
Settings used

- Q = 0.00001 per day
- R = 1 (spread noise)
- first guess β = 0, P = 1

A wrong β turns a stable spread into a drifting one

Picture of the model: EWC tracks β times EWA, and β itself can move





A pairs trade needs a hedge ratio that may change over time

Basic knowledge — EWA (Australia ETF) and EWC (Canada ETF) both track commodity exporters

Buy 1 share of EWC and sell β shares of EWA.
The spread between them should keep returning to its mean.

Why β matters

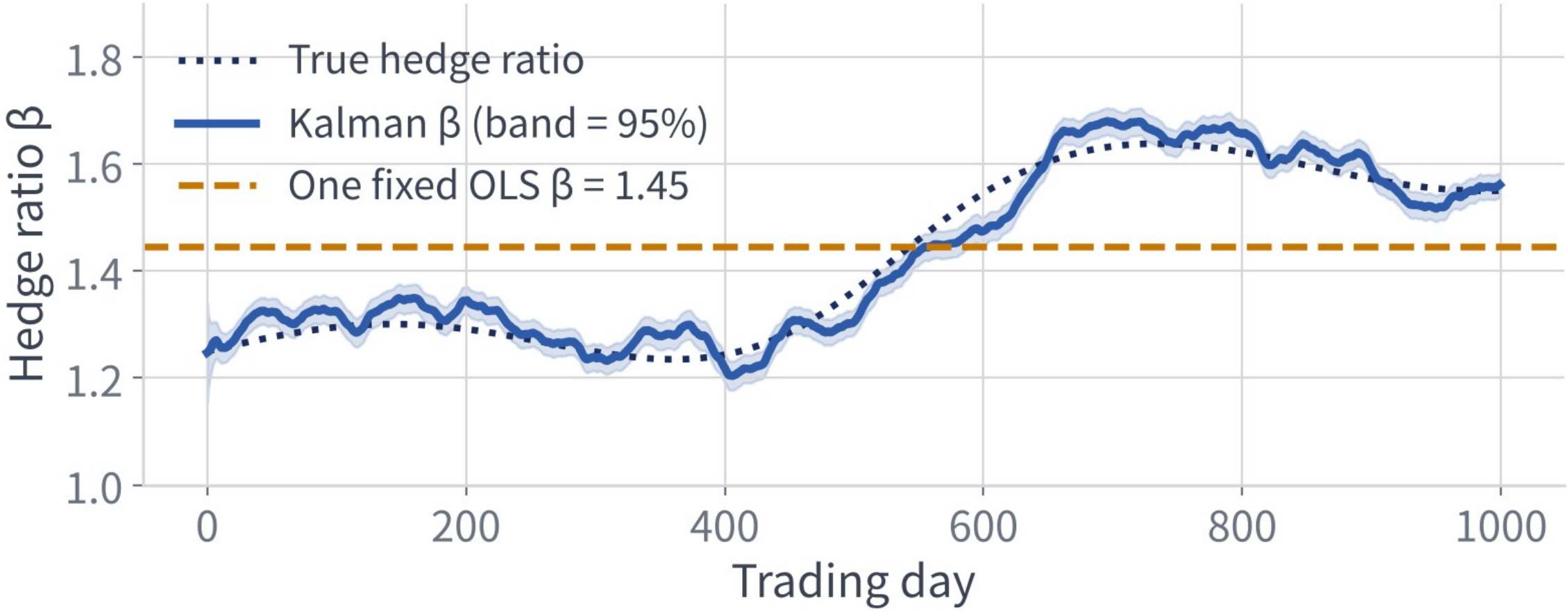
- Wrong β: the spread drifts
- Drifting spread: false signals
- β changes as economies change

The Kalman answer

- Treat β as the hidden state
- Update it every day
- Use the innovation as signal

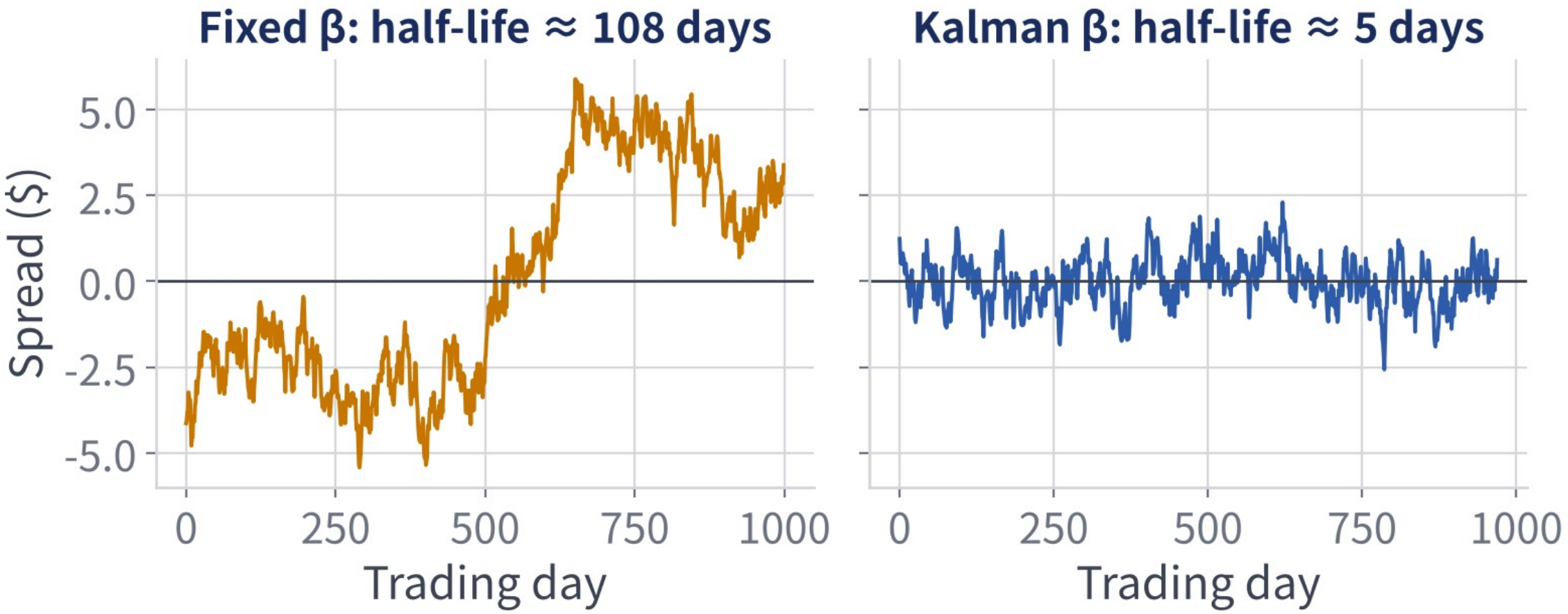
The Kalman hedge ratio follows the true β; one fixed β cannot

Daily filter on EWA-like and EWC-like prices · simulated data · Q = 0.00001, R = 1



With the Kalman β the spread keeps returning to zero

Spread is EWC minus β times EWA, using yesterday's β so no future data enter · simulated data

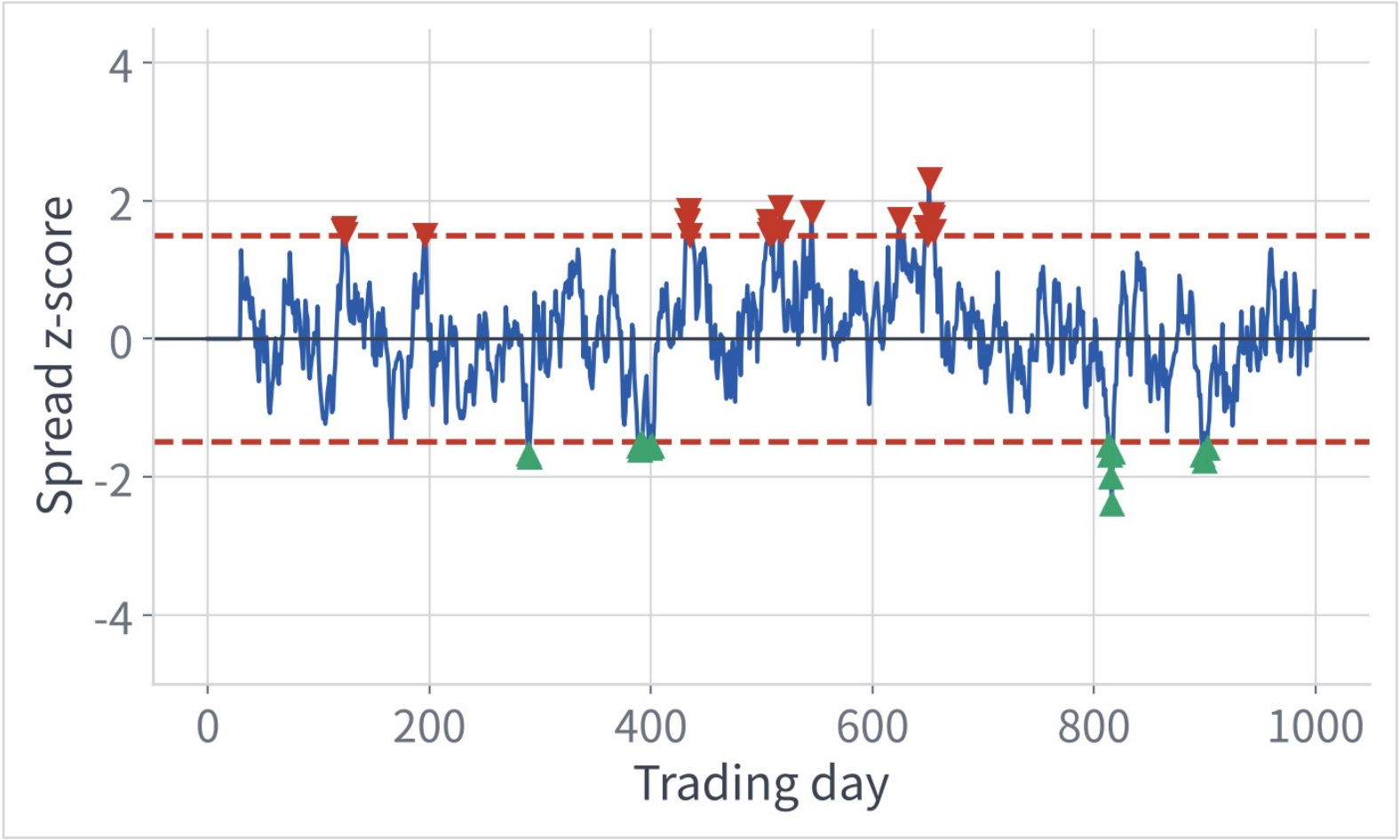


The innovation z-score is a ready-made trading signal

Red triangles: sell the spread above +1.5 · green triangles: buy below −1.5

Why it works

- Innovation = surprise
- Its variance is known
- So the z-score is unit-free
- Costs not included



Stock beta: the state is alpha and beta, H holds the market return

Model first — time-varying CAPM regression in state-space form

$$\text{State equation}: \quad \begin{bmatrix} \alpha \\ \beta \end{bmatrix}_k = \begin{bmatrix} \alpha \\ \beta \end{bmatrix}_{k-1} + w_{k-1}$$

$$\text{Measurement equation}: \quad r_k^{\text{stock}} = \begin{bmatrix} 1 & r_k^{\text{mkt}} \end{bmatrix} \begin{bmatrix} \alpha \\ \beta \end{bmatrix}_k + v_k$$

Slots

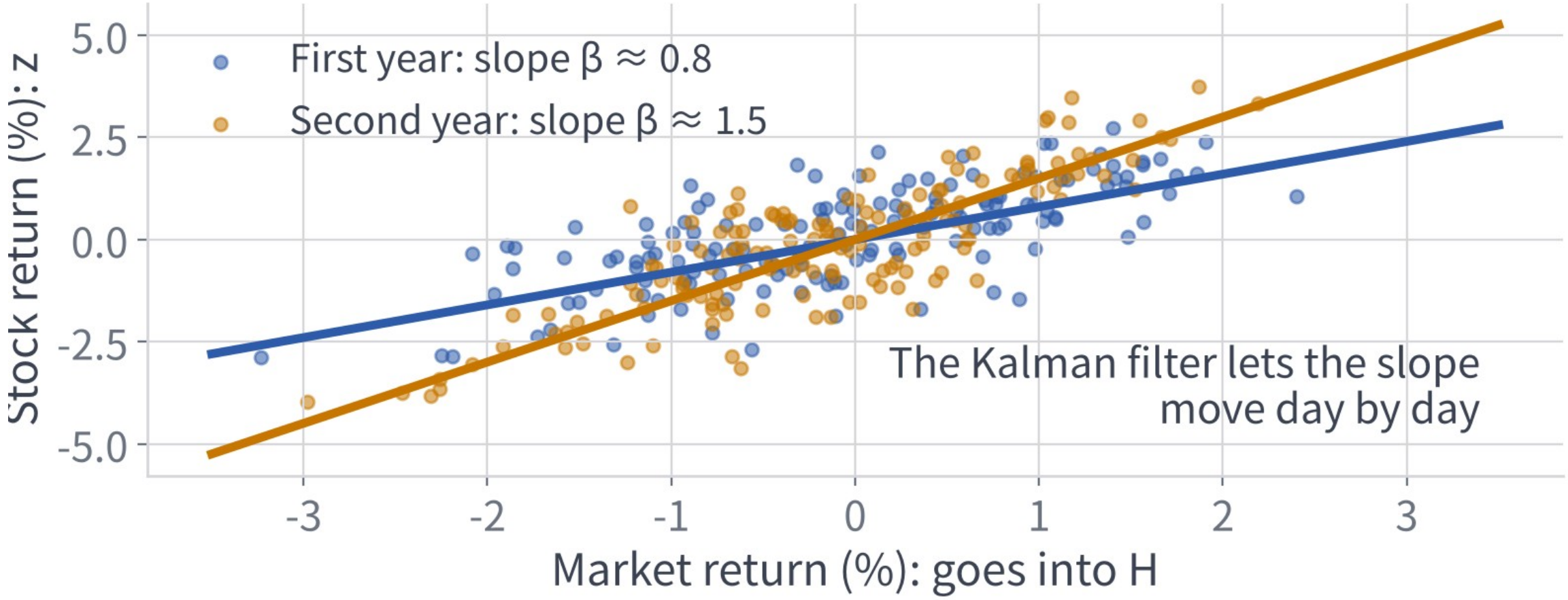
- x: α and β
- A = 1: both drift slowly
- H: [1, market return]

Settings used

- Q: β moves faster than α
- R = daily residual variance
- H changes every day

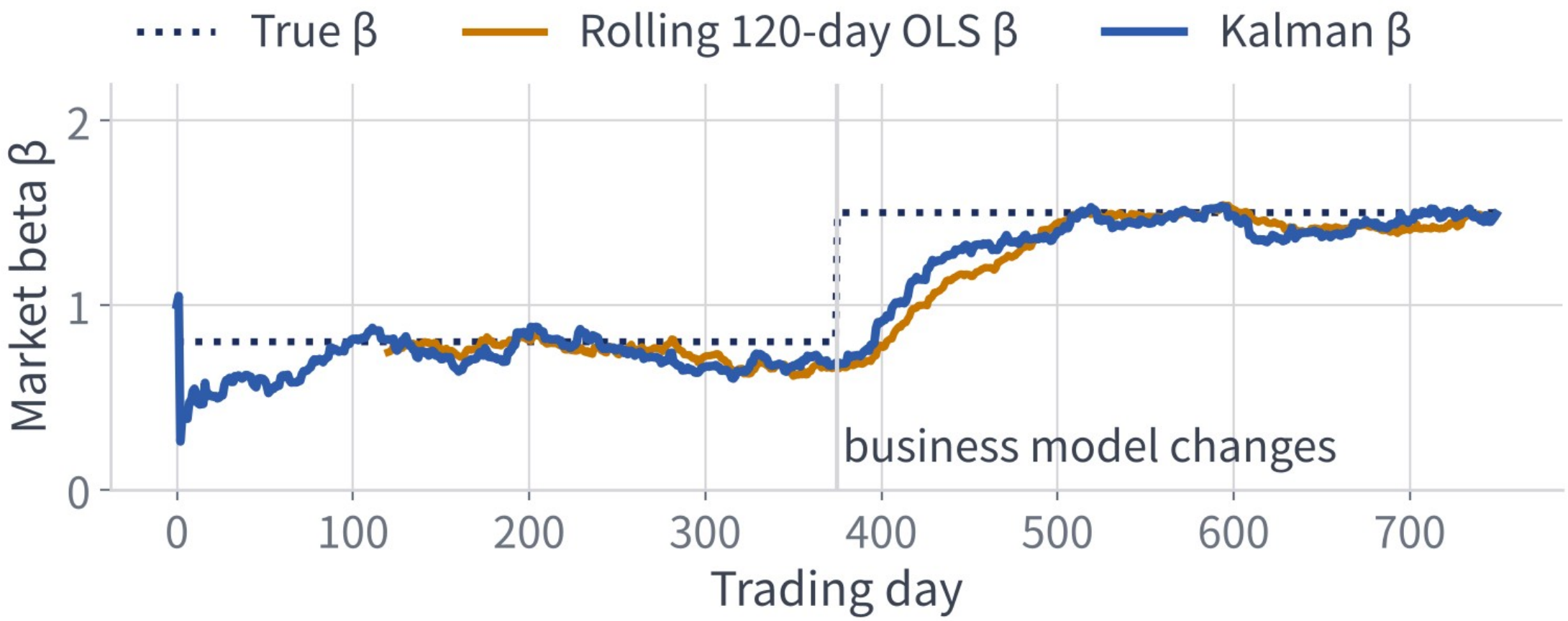
Beta is the slope of stock returns on market returns

Picture of the model: H is the market return, z is the stock return, β is the slope



The Kalman beta adapts to a regime change without a window

True β jumps from 0.8 to 1.5 on day 375 · daily returns · simulated data



Nowcasting: the growth gap G is the state; each release measures it with error

Model first — use the gap G = g − μ so the template x = A x + w applies

$$\text{State}: \quad \underbrace{G_k}_{\text{gap today}} = \underbrace{\phi}_{\text{A}} G_{k-1} + \underbrace{w_{k-1}}_{\text{new shock}}$$

$$\text{Measurement}: \quad z_k = \underbrace{1}_{\text{H}} \cdot G_k + \underbrace{v_k}_{\text{release error}}$$

$$\text{where}: \quad G_k = g_k - \mu \ \ (\text{true growth minus its average}), \quad z_k = \text{published growth} - \mu$$

Slots

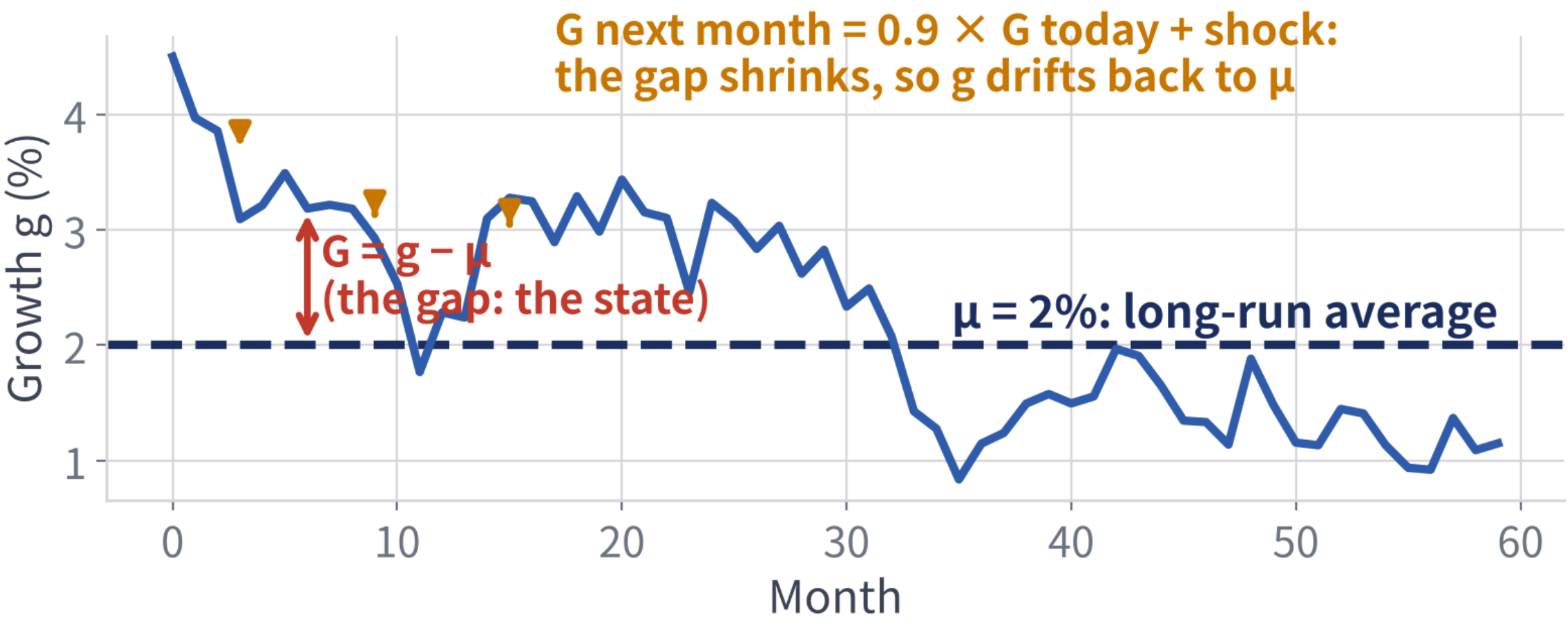
- x = G, the growth gap
- A = φ = 0.9, H = 1

Settings used

- μ = 2%, Q = 0.25, R = 0.36
- v: error in the release

μ is the level that growth keeps returning to

μ = long-run average growth; the state G is the distance of g from μ



The gap G is predicted every month but corrected only when data are published

Both rows refer to the state G = g − μ; z is the published growth minus μ



Both rows are about the gap G = g − μ. G is predicted every month;

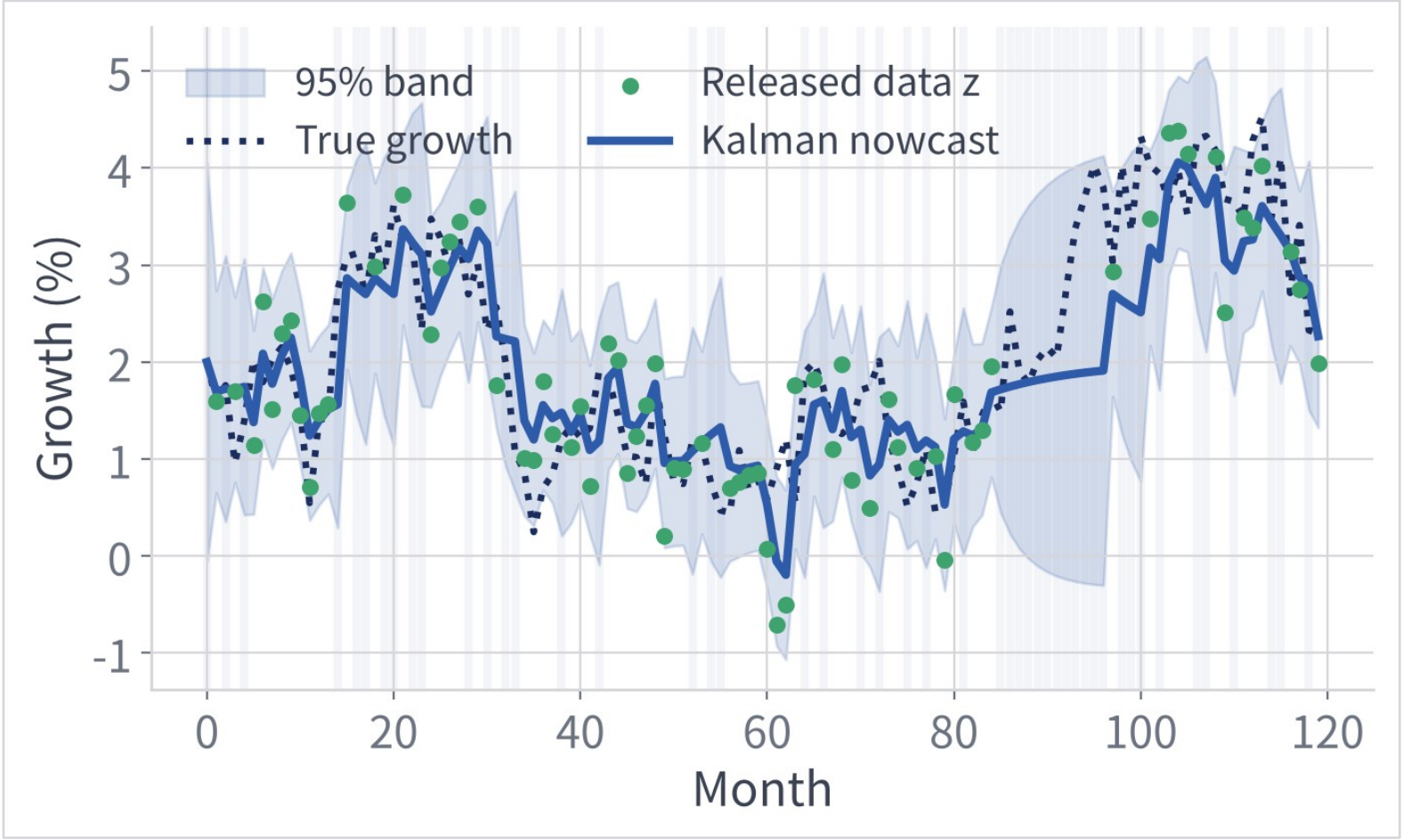
it is corrected only in months whose number z is published. v = error in that number.

Missing releases are handled by skipping the update

Filtered G plus μ, drawn as growth g — official data arrive late and irregularly

Rule for gaps

- Data present: update
- Data missing: predict only
- Band widens in gaps
- Central banks use this



Several monthly series and quarterly GDP share one hidden growth gap

Mixed-frequency dynamic factor nowcast; loadings are illustrative

$$\text{State}: \quad G_k = \phi\, G_{k-1} + w_{k-1} \qquad (G_k: \text{ common growth gap})$$

$$\text{Measurement}: \quad \begin{bmatrix} z^{\text{IP}} \\ z^{\text{Retail}} \\ z^{\text{Jobs}} \\ z^{\text{GDP}} \end{bmatrix}_k = \underbrace{\begin{bmatrix} 1.0 \\ 0.8 \\ 0.6 \\ 0.9 \end{bmatrix}}_{\text{H (loadings)}} G_k + \begin{bmatrix} v^{\text{IP}} \\ v^{\text{Retail}} \\ v^{\text{Jobs}} \\ v^{\text{GDP}} \end{bmatrix}_k$$

Slots

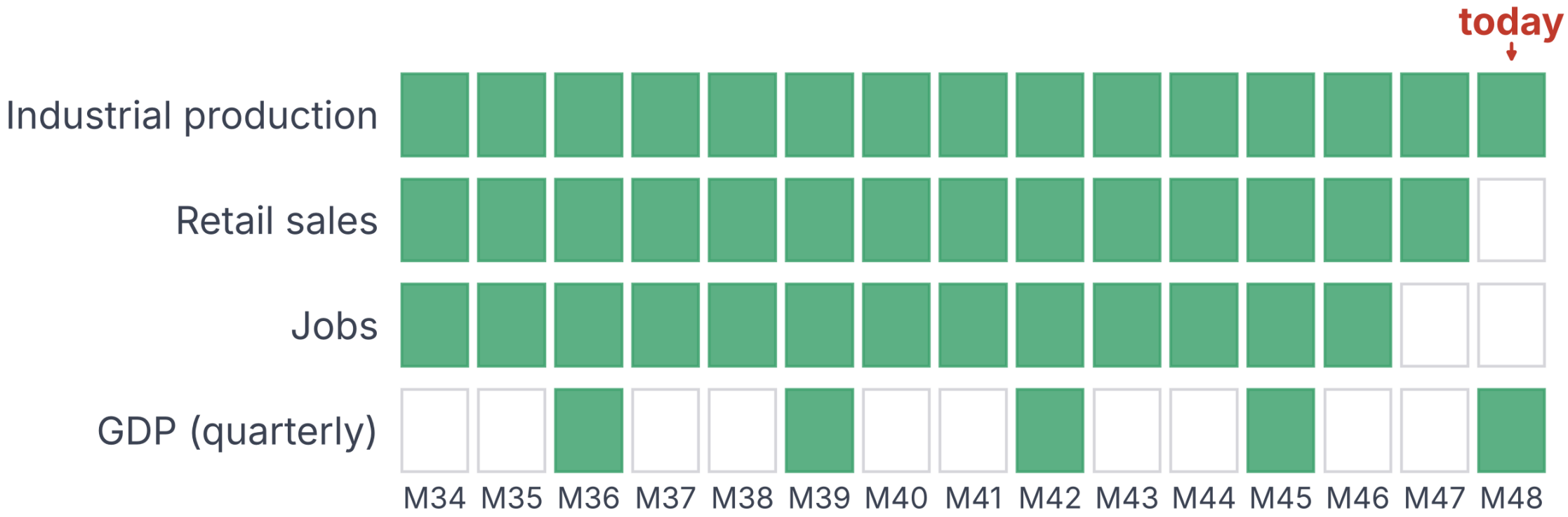
- x = G, one common gap
- H: one loading per series

Mixed frequency

- Monthly series: every month
- GDP: third month of a quarter

Each month the filter uses only the rows that were released

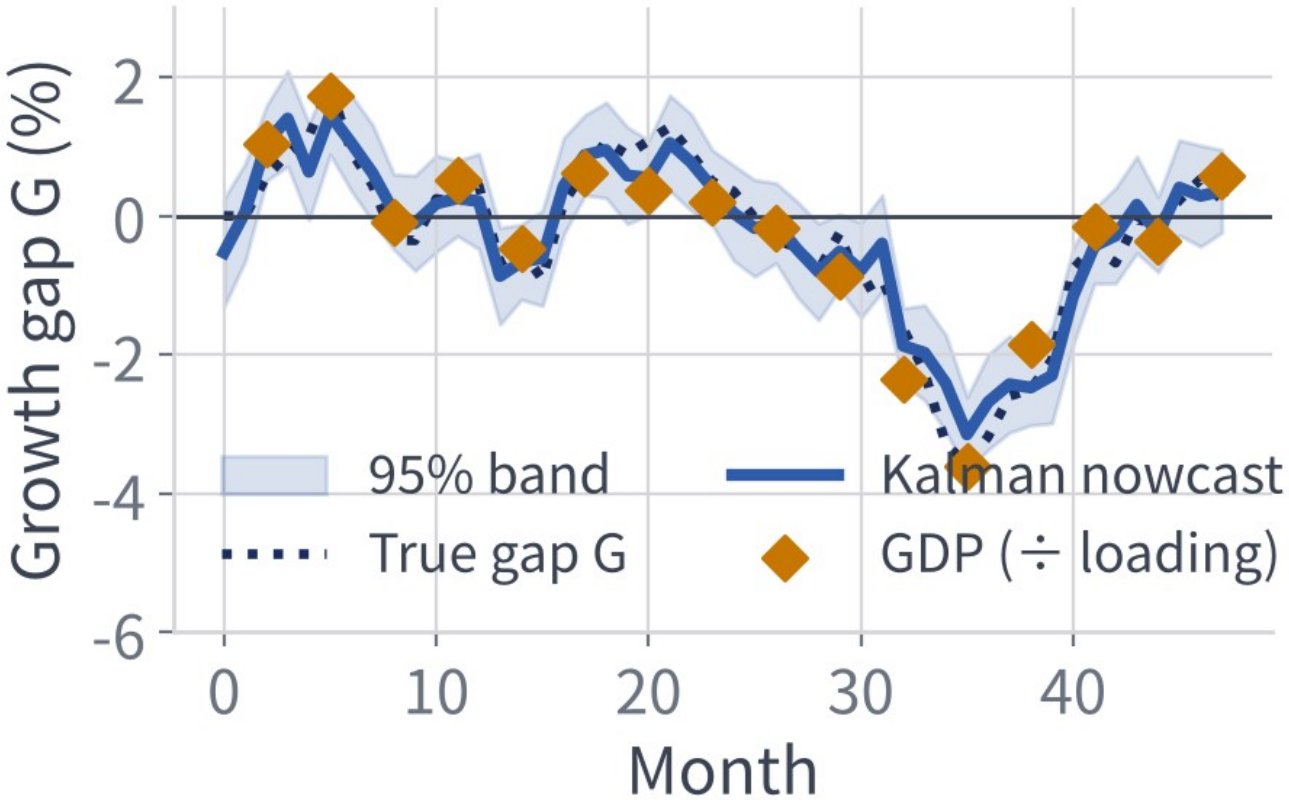
Ragged edge: the newest month has only industrial production and GDP; jobs and retail sales come later

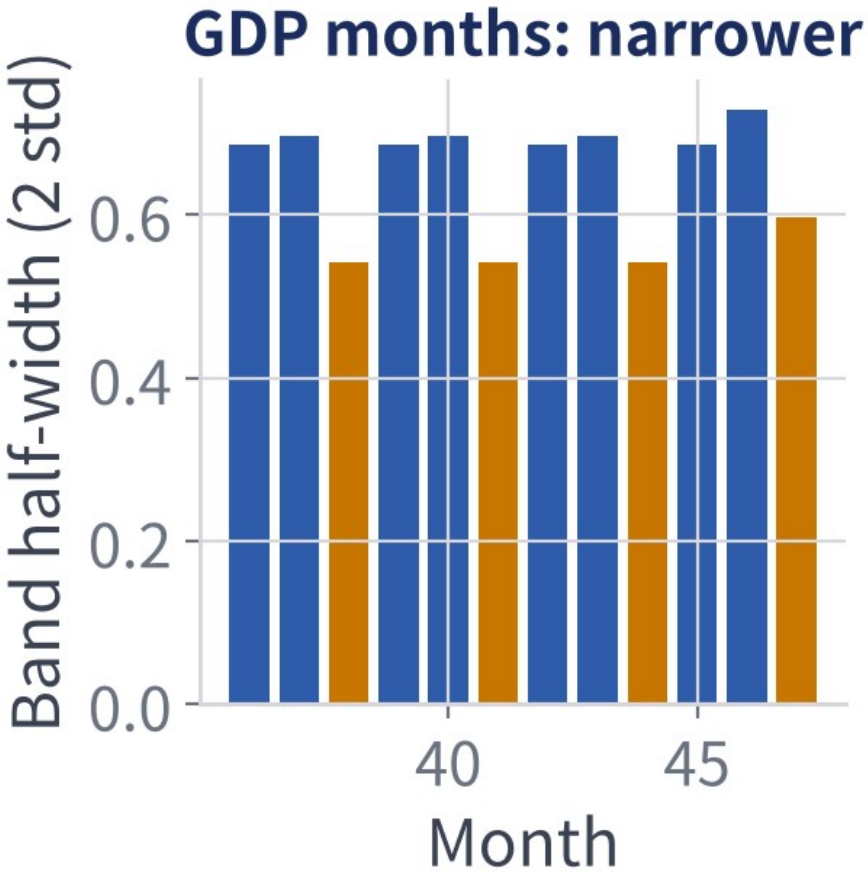


Green = number released that month.
Each month H keeps only the green rows.

Combining all series tracks the gap even between GDP releases

Simulated data · GDP is treated as a reading of G in the quarter's last month (a simplification)





Yield curve: the three factor gaps are the state, their shapes form H

Model first — use the gaps F = factor − μ so the template x = A x + w applies

$$\text{State}: \quad F_k = \underbrace{A}_{0.97 \text{ each}} F_{k-1} + w_{k-1}$$

$$\text{Measurement}: \quad z_k(\tau) = \underbrace{\begin{bmatrix} 1 & \lambda_S(\tau) & \lambda_C(\tau) \end{bmatrix}}_{\text{H}} F_k + v_k$$

$$\text{where}: \quad F_k = \begin{bmatrix} L_k - \mu_L \\ S_k - \mu_S \\ C_k - \mu_C \end{bmatrix} \text{ (factor minus its average)}, \quad z_k(\tau) = y_k(\tau) - \bar{y}(\tau)$$

Slots

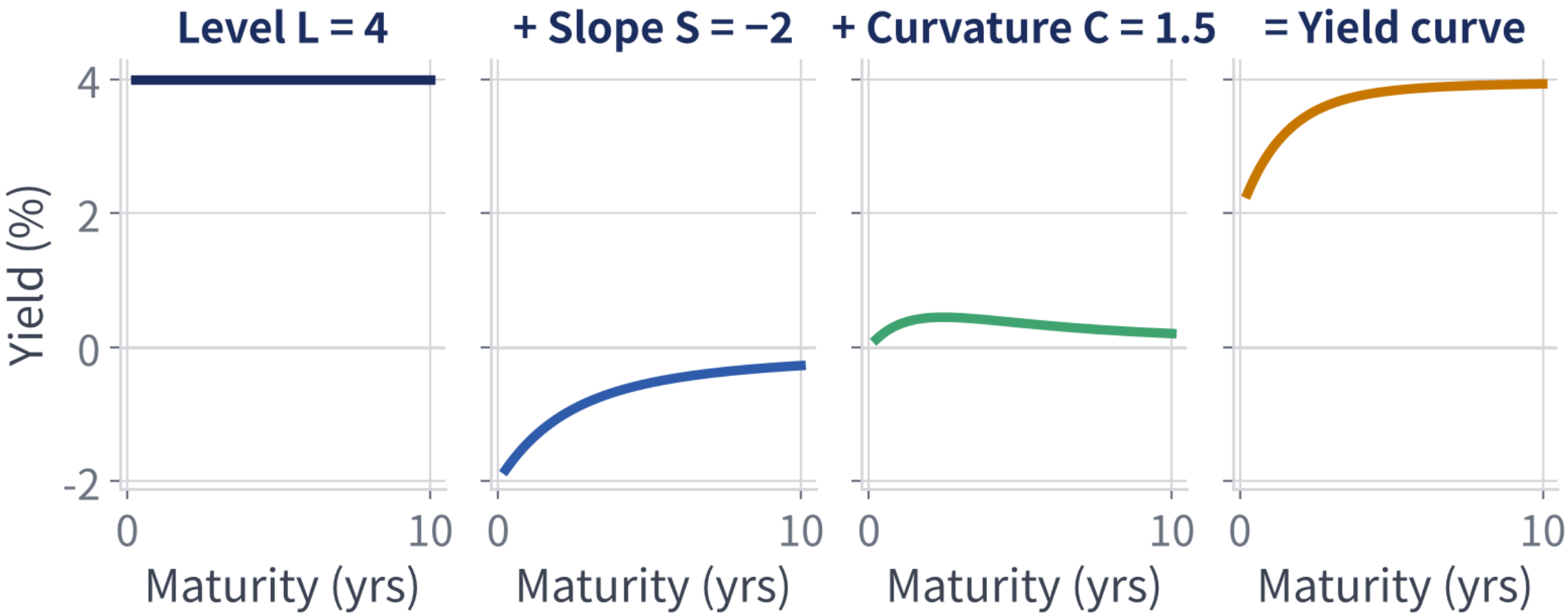
- x = F: level, slope, curvature gaps
- A = 0.97: each gap shrinks

Settings used

- H: λ(τ), one row per maturity
- v: fitting error per yield

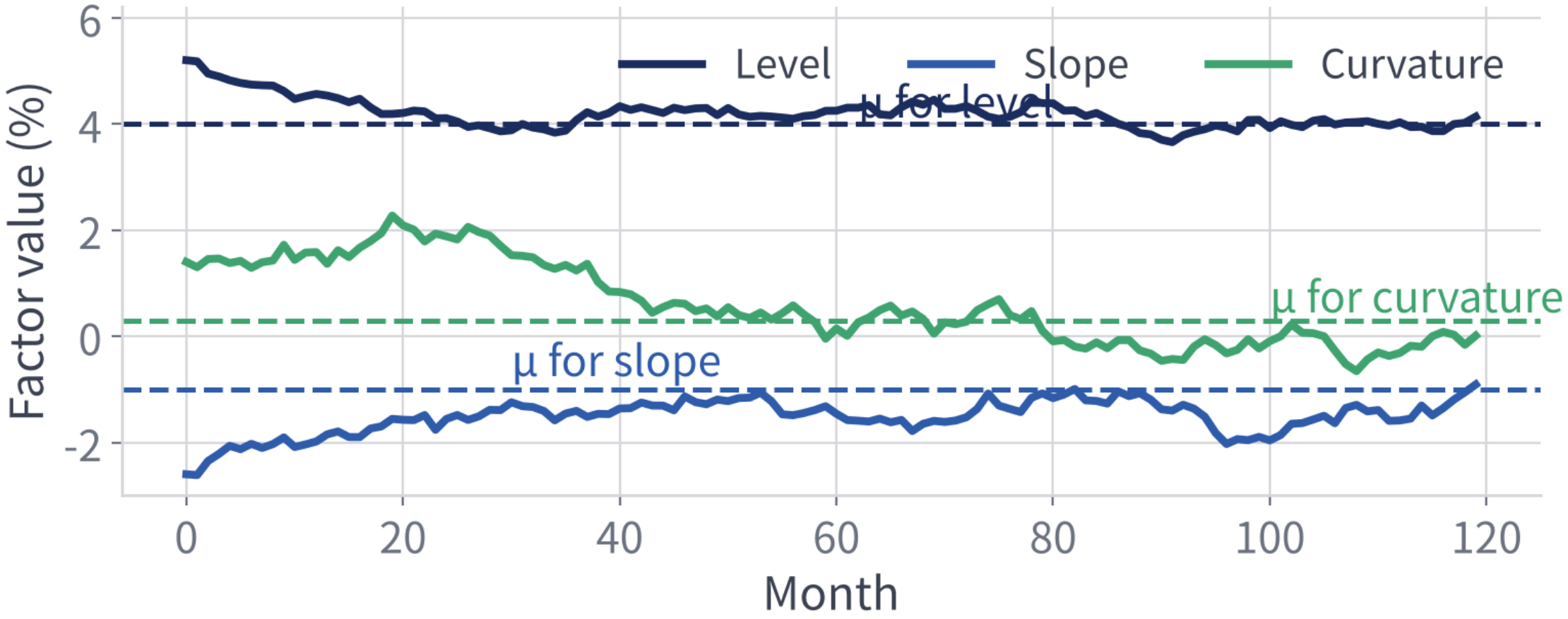
Three factors times three fixed shapes rebuild the whole yield curve

Measurement equation in a picture: the loadings λ(τ) are the rows of H



Each factor has its own μ and drifts back toward it

State equation in a picture: A = 0.97 closes 3% of the gap to μ every month

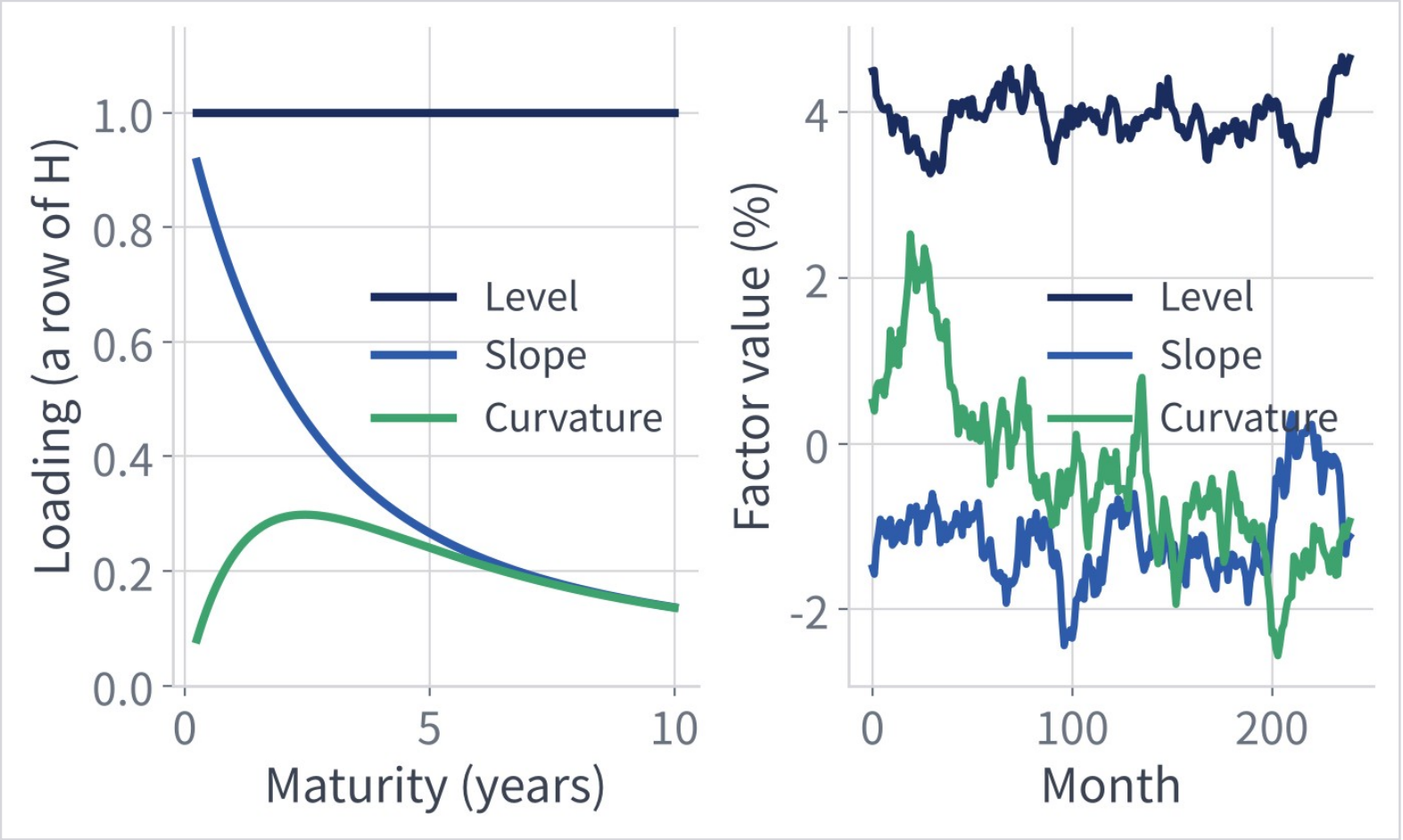


The whole yield curve is summarised by three filtered factors

Dynamic Nelson–Siegel model — loadings form H, factors form the state x

Slots

- z = yields at all maturities
- x = level, slope, curvature
- H = loadings (left panel)
- A: factors move slowly



Outside finance the same filter runs in everyday devices

Examples of the template in engineering and science

Field	Hidden state x	Measurement z
Aircraft and drones	attitude, position	gyros, GPS
Smartphones	location, heading	GPS, accelerometer, compass
Video tracking	box position and speed	detector boxes per frame
Weather forecasting	atmosphere state	satellites, stations
Wearables	heart rate	noisy optical sensor

06

BEYOND THE BASICS

Extensions in Pictures

Smoothing, learning Q and R, and nonlinear models

A smoother is a replay that re-judges the past with later data

Basic knowledge — filtering asks 'where am I now?'; smoothing asks 'where was I back then?'

Filter = live commentator: judges each moment as it happens.
Smoother = replay analyst: re-judges each moment after the game.

Everyday example

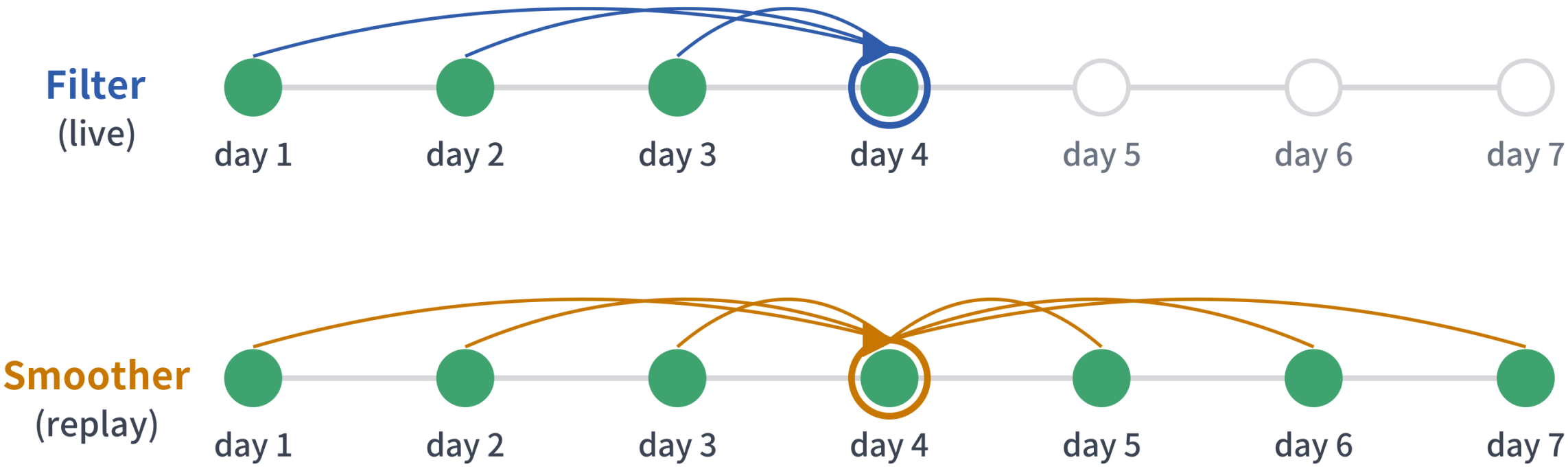
- Phone GPS jumps in a tunnel
- After the trip the route is redrawn
- Where you came out fixes the path

Why it is more accurate

- Each day uses more data
- Clues come from both sides
- But only after the fact

The filter looks only backward; the smoother looks both ways

Which days each method may use when it estimates day 4



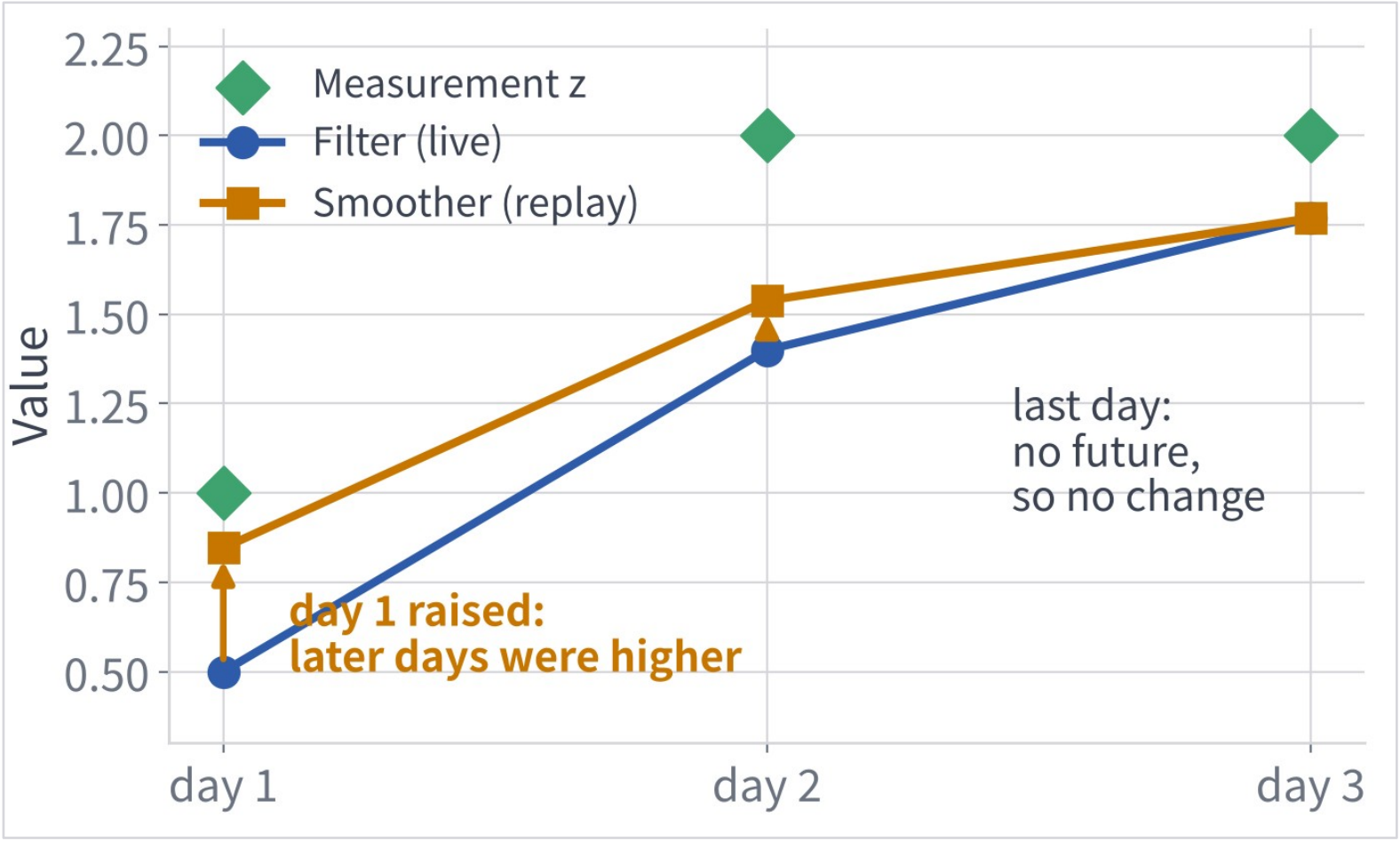
Estimating day 4: the filter may use days 1–4; the smoother may use every day

With three data points the smoother pulls early estimates toward later data

Data 1, 2, 2 with Q = R = 1 — the same numbers as the EM worked example

What changed

- Day 1: 0.50 → 0.85
- Day 2: 1.40 → 1.54
- Day 3: no change
- Later data were higher



The backward pass corrects each day by how much the next day was revised

Start on the last day, where filter and smoother agree, and walk back one day at a time

$$\hat{x}_{k|T} = \underbrace{\hat{x}_{k|k}}_{\text{filter estimate today}} + \underbrace{C_k}_{\text{weight}} \Big(\underbrace{\hat{x}_{k+1|T} - \hat{x}^{-}_{k+1}}_{\text{how much tomorrow was revised}} \Big)$$

$$C_k = \frac{P_{k|k}\, A}{P^{-}_{k+1}} = \frac{\text{uncertainty today (filter)}}{\text{uncertainty predicted for tomorrow}} \times A$$

The weight C

- Large when today was uncertain
- Small when today was already sure

Rule of use

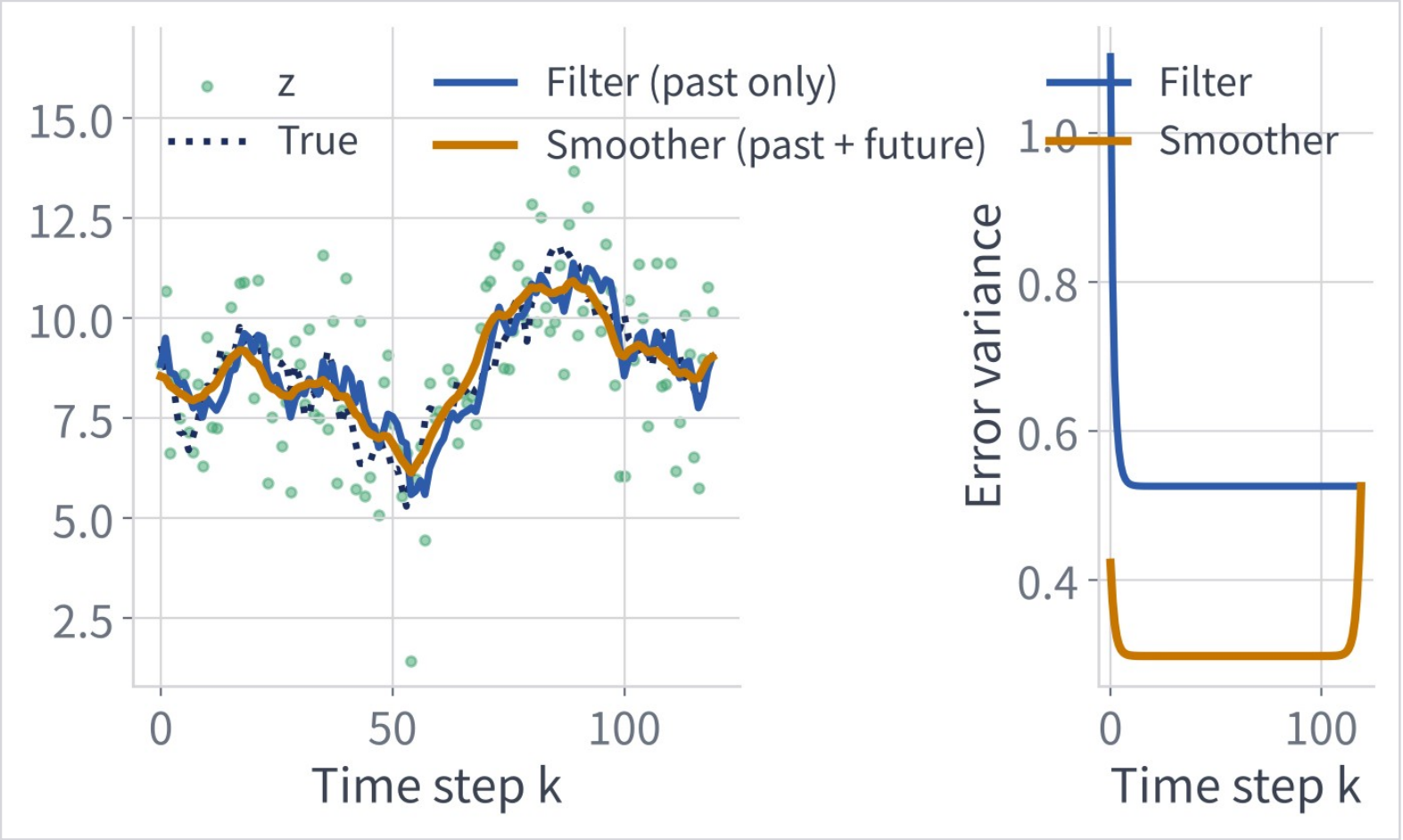
- Fine for analysis, charts and EM
- Never as a live trading signal

Looking back with future data gives a smoother path

Kalman smoother (Rauch–Tung–Striebel) — run the filter forward, then one pass backward

Use with care

- Filter: past data only
- Smoother: past and future
- Lower error variance
- Never in a backtest signal

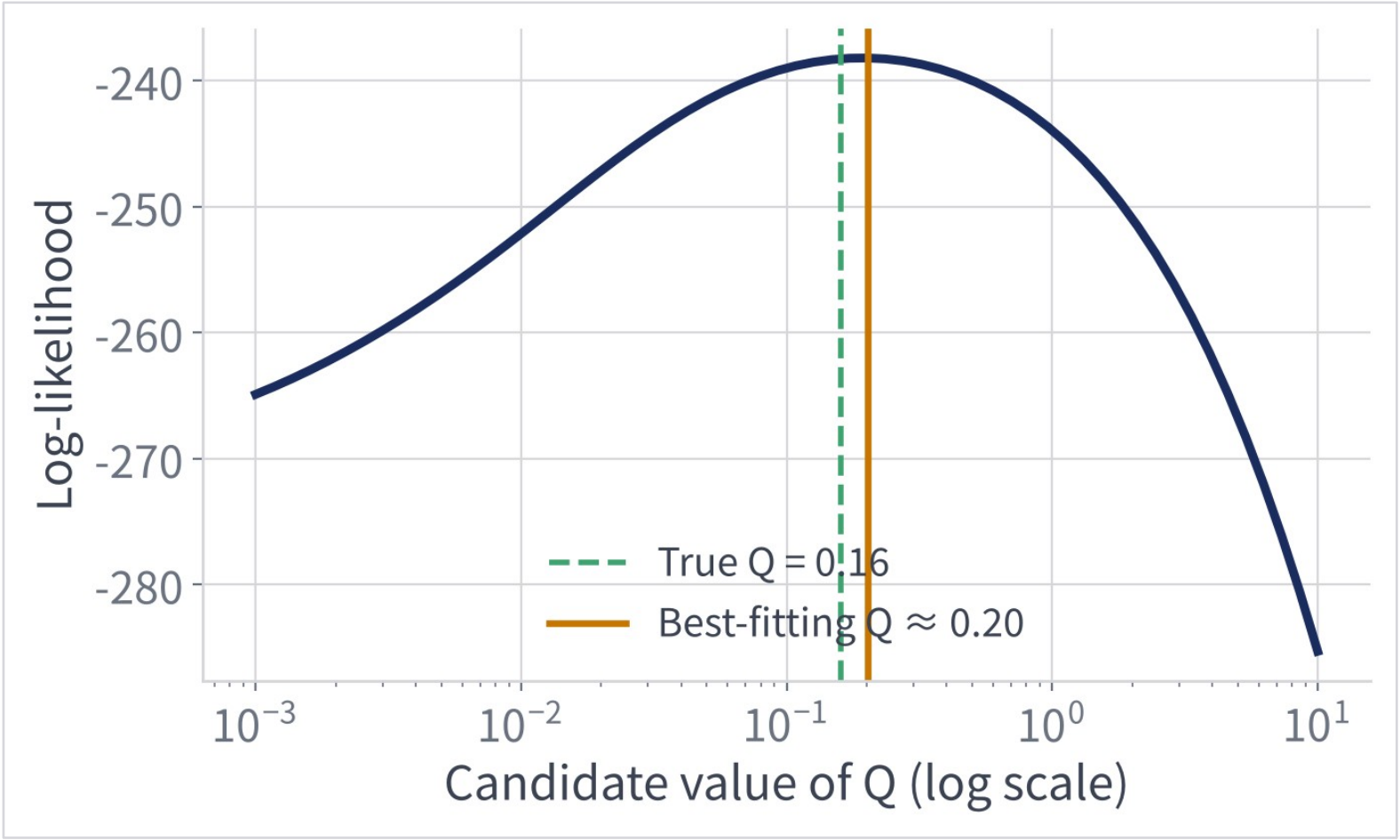


Q and R can be estimated from the innovations themselves

Maximum likelihood — pick the Q that makes the surprises look most like their predicted size

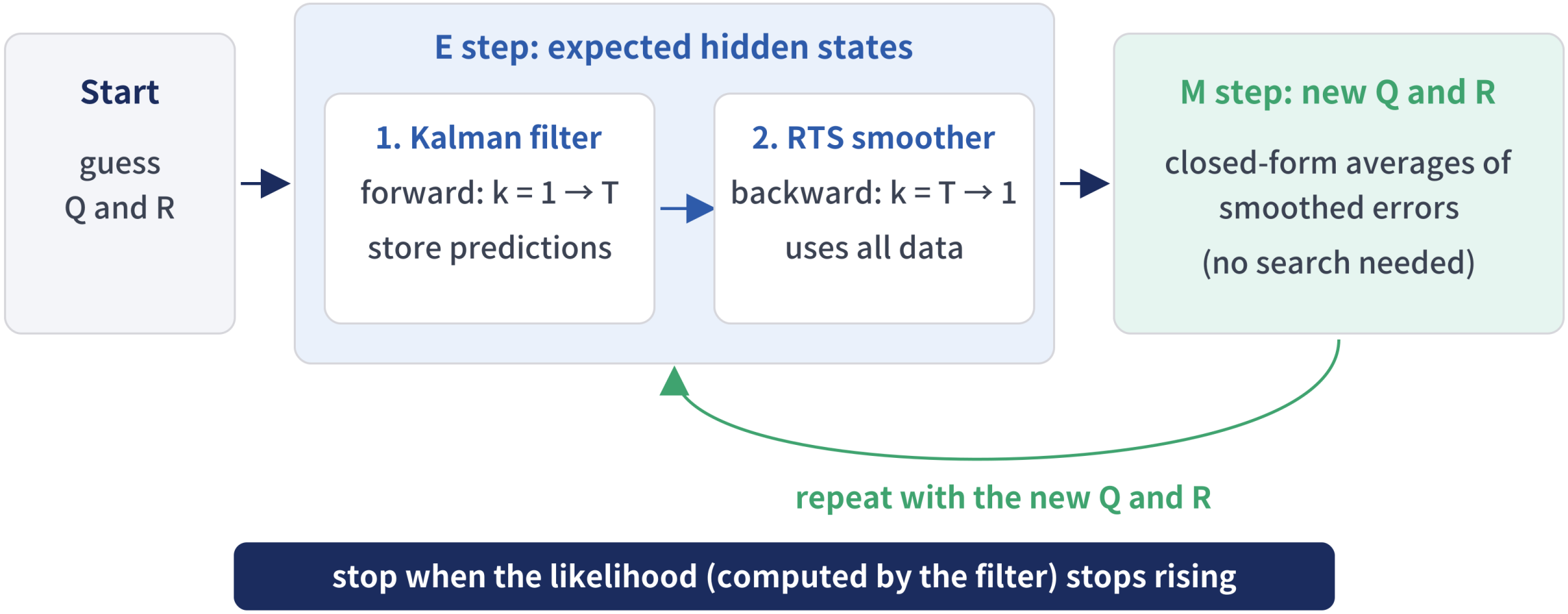
Idea

- Each Q gives innovations
- Score how well they fit
- Pick the best score
- EM is an alternative



In EM the E step is the Kalman smoother, not the filter alone

Expectation–maximization (Shumway–Stoffer, 1982) — the filter runs forward, the smoother runs back



The M step turns smoothed estimates into new Q and R in closed form

Scalar local level model (A = H = 1); the subscript k|T means 'given all T data points'

$$\hat{R} = \frac{1}{T}\sum_{k=1}^{T}\Big[\underbrace{(z_k - \hat{x}_{k|T})^2}_{\text{fit error}} + \underbrace{P_{k|T}}_{\text{state still uncertain}}\Big]$$

$$\hat{Q} = \frac{1}{T-1}\sum_{k=2}^{T}\Big[\underbrace{(\hat{x}_{k|T} - \hat{x}_{k-1|T})^2}_{\text{smoothed step}} + P_{k|T} + P_{k-1|T} - 2P_{k,k-1|T}\Big]$$

Why the smoother

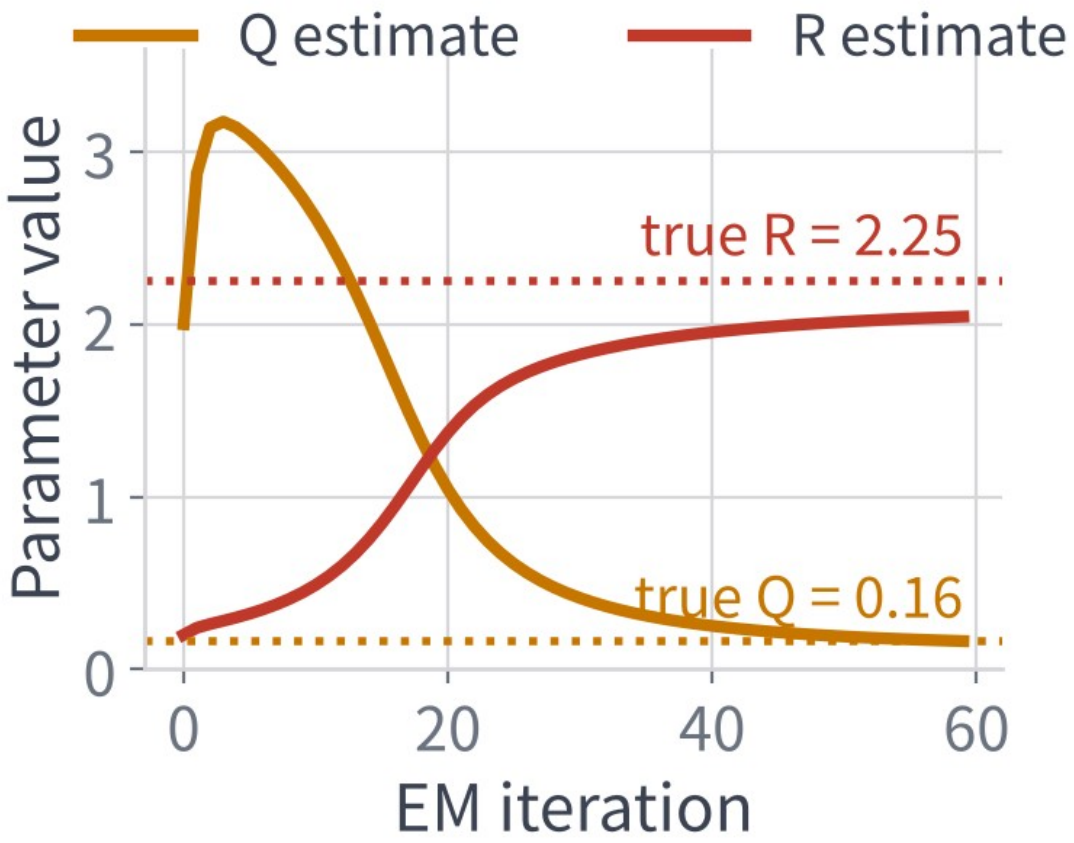
- Needs states given all data
- The filter sees only the past

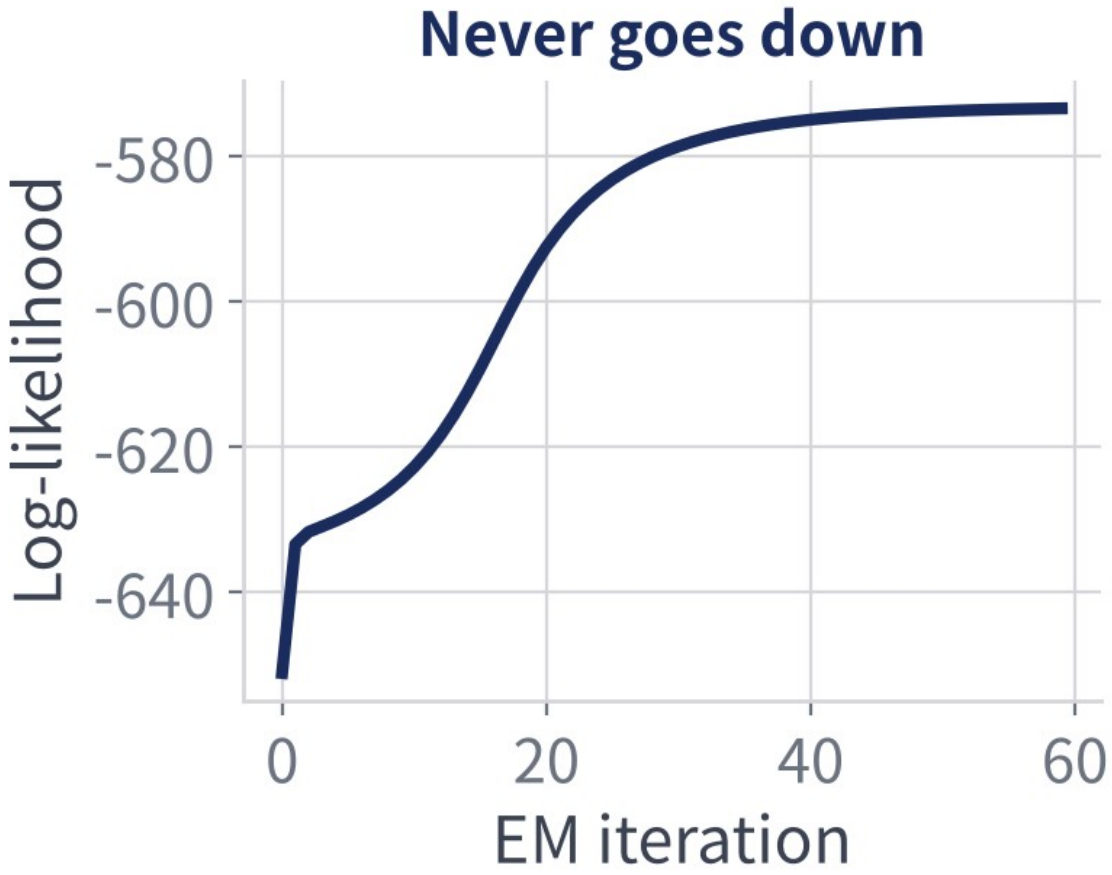
Do not drop P

- P terms: the state is uncertain
- Without them R is too small

Starting from a poor guess, EM walks Q and R to the true values

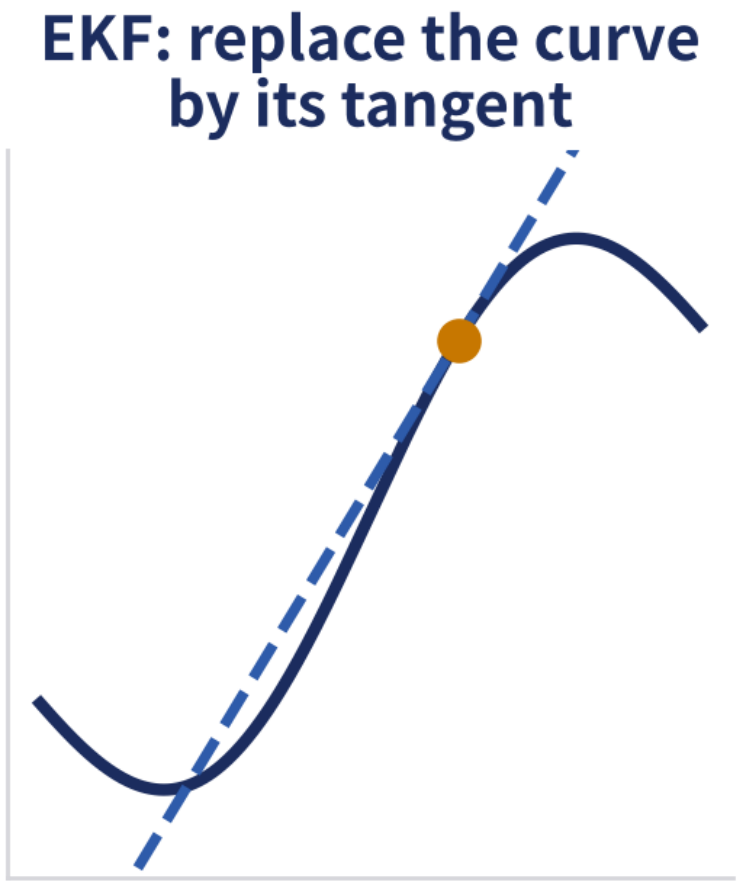
Simulated data with true Q = 0.16, R = 2.25; start at Q = 2.0, R = 0.2

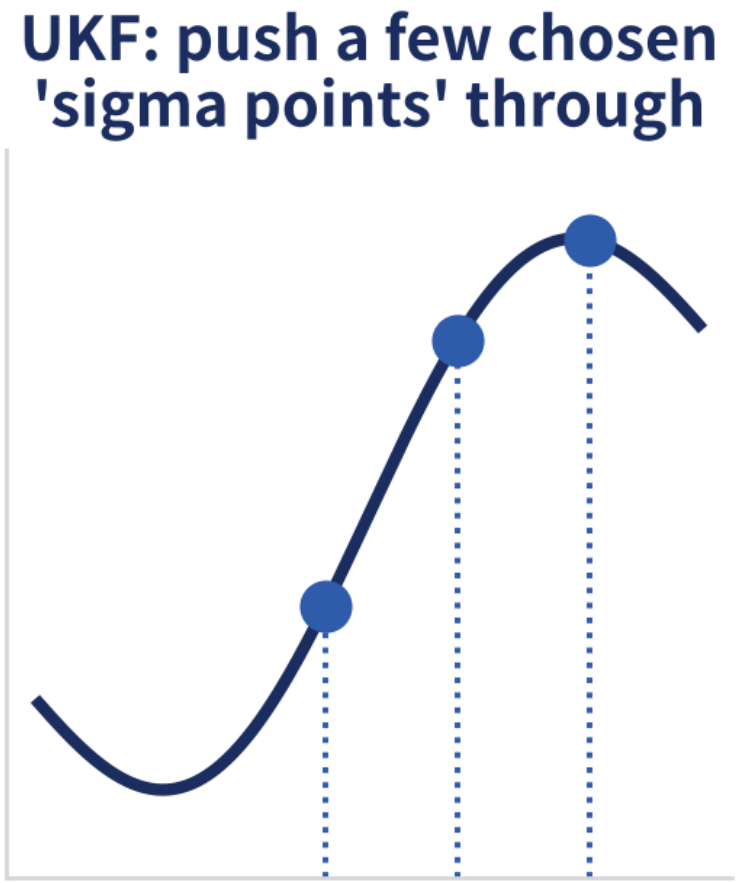


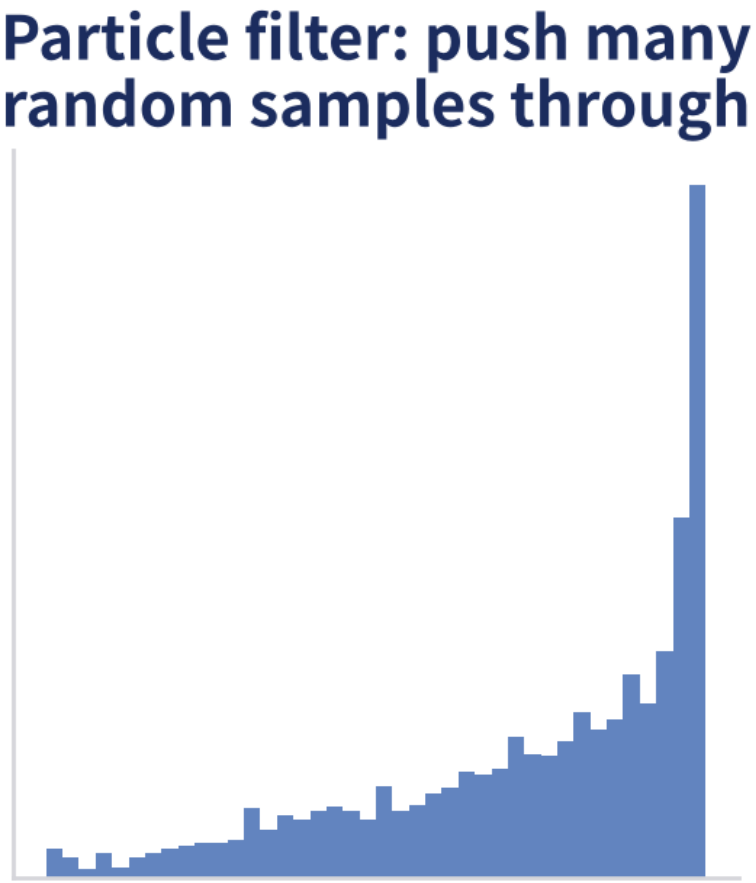


When the model is curved, three filters approximate it differently

Extended (EKF), unscented (UKF) and particle filter — same predict-update loop







The problem's shape decides which filter to use

Start with the basic filter; move down the table only when it clearly fails

Situation	Filter	Example
Linear model, normal noise	Kalman filter	hedge ratio, trend
Mildly curved model	EKF	interest-rate models
Strongly curved model	UKF	option-implied states
Fat tails or jumps	Particle filter	stochastic volatility
Switching regimes	Regime-switching filter	calm vs crisis

Six mistakes cause most bad Kalman results

Check these before trusting the output

Failures come from a wrong model or a wrong use of the output, almost never from the five equations.

Model mistakes

- Q too small: lags behind breaks
- Q too large: chases noise
- Wrong H: estimate drifts

Use mistakes

- Smoother values in a backtest
- Trusting the first few steps
- Ignoring the error band P

The Kalman filter is one template filled in many ways

What to remember

1

Blend, do not choose

The estimate is a weighted average of prediction and measurement; the gain K is the weight.

2

Two equations, two steps

State and measurement equations; predict and update — repeated at every new data point.

3

Applications are slot-filling

Decide x, A, z, H, Q and R. Tracking, hedge ratios, beta, nowcasting and yield curves all follow.

Always report the error band P — the filter knows how unsure it is.

Questions for discussion

Answer with the ideas in this deck — no calculation needed

Q1	A GPS unit becomes twice as noisy. Which of Q or R changes, and does the gain rise or fall?	**Gain**
Q2	Your Kalman hedge ratio jumps around daily. What would you change first, and what do you give up?	**Tuning**
Q3	A backtest uses smoothed β and looks excellent. Why should you distrust it?	**Smoothing**
Q4	A monthly indicator is missing for three months. What does the filter do, and how does P behave?	**Missing data**

A

APPENDIX

Four Calculations by Hand

Two filter steps, a two-sensor update and an M step, line by line

Step one moves the estimate from 10 to 12.4

Last estimate 10 with P = 3; A = 1, Q = 1, R = 1; measurement z = 13

$$\text{Predict}: \quad \hat{x}^{-} = 1 \times 10 = 10, \qquad P^{-} = 1 \times 3 \times 1 + 1 = 4$$

$$\text{Gain}: \quad K = \frac{4}{4+1} = 0.8$$

$$\text{Update}: \quad \hat{x} = 10 + 0.8\,(13 - 10) = 12.4, \qquad P = (1 - 0.8) \times 4 = 0.8$$

Check

- Matches the bell-curve slide
- P falls from 4 to 0.8

Meaning

- K = 0.8: trust GPS more
- 80% of the surprise is used

Step two uses a smaller gain because the filter is more certain

Same model; next measurement z = 12

$$\text{Predict}: \quad \hat{x}^{-} = 12.4, \qquad P^{-} = 0.8 + 1 = 1.8$$

$$\text{Gain}: \quad K = \frac{1.8}{1.8 + 1} \approx 0.64$$

$$\text{Update}: \quad \hat{x} = 12.4 + 0.64\,(12 - 12.4) \approx 12.14, \qquad P \approx 0.64$$

Check

- K falls from 0.8 to 0.64
- P falls to about 0.64

Pattern

- Each step K moves toward its steady value
- Same as the gain chart in Python

Two sensors at once: the vector gain splits the trust

Sensor fusion example — prediction 100 with P = 4; GPS reads 96, the wheel sensor reads 102

$$\begin{aligned}
\text{Set up}: &\quad \hat{x}^- = 100,\ P^- = 4,\ R_1 = 9,\ R_2 = 2.25,\ z_1 = 96,\ z_2 = 102 \\
\text{Gain}: &\quad D = 4 \times 11.25 + 9 \times 2.25 = 65.25, \qquad K_1 = \tfrac{4\times 2.25}{65.25} \approx 0.14, \qquad K_2 = \tfrac{4\times 9}{65.25} \approx 0.55 \\
\text{Update}: &\quad \hat{x} = 100 + 0.14\,(-4) + 0.55\,(+2) \approx 100.55, \qquad P = (1 - 0.69) \times 4 \approx 1.24
\end{aligned}$$

Check

- $K_1 \approx 0.14$ for noisy GPS
- $K_2 \approx 0.55$ for the wheel sensor

Meaning

- P falls from 4 to 1.24
- Lower than with either sensor alone

One M step for R uses the smoothed states and their variances

Data 1, 2, 2 with Q = R = 1; the E step (filter + smoother) gives line one

$$\text{E step}: \quad \hat{x}_{k|T} = (0.846,\ 1.538,\ 1.769), \qquad P_{k|T} = (0.385,\ 0.462,\ 0.615)$$

$$\text{M step}: \quad \hat{R} = \tfrac{1}{3}\Big[(0.154^2 + 0.385) + (0.462^2 + 0.462) + (0.231^2 + 0.615)\Big]$$

$$= \tfrac{1}{3}\Big[0.409 + 0.675 + 0.668\Big] \approx 0.58$$

Check

- Fit errors alone give 0.10
- Adding P lifts it to 0.58

Next

- Rerun the E step with the new R
- Repeat until the likelihood is flat